

Agent Architectures

From Generative Model to Goal-Directed System

INTENDED AUDIENCE

Beginners who want structural literacy before specializing. The book is designed for self-learners, students, technical readers, and practitioners who want to understand how agentic systems are put together without first committing to a framework or production stack.

SCOPE

This book is a conceptual textbook on the architectural spine of agentic systems: how a generative model becomes a bounded, goal-directed system through explicit goals, control loops, state, memory, planning, action selection, tool interfaces, grounding, feedback, reflection, stopping, and handoff. It uses a bounded literature-review assistant as its primary recurring case and treats architecture choice as driven by task shape.

Detailed Table of Contents

Part I — Architectural Foundations

Establish the difference between a model and an agent, then build the representational backbone of goals, state, context, and memory.

1. Chapter 1 — From Generative Model to Agentic System

Distinguish a bare generative model from interactive systems, scripted workflows, and agentic loops, using the literature-review assistant as the running comparison.

2. Chapter 2 — Goals, Task Specifications, and Success Criteria

Turn requests into operational task specifications that can steer control, evaluation, and stopping.

3. Chapter 3 — Loops, State, Context, and Execution Traces

Make information placement visible across time and show why context is not durable state.

4. Chapter 4 — Memory Types, Persistence, and Policy

Treat memory as representation plus policy rather than as a generic database or a bigger context window.

Part II — Deliberation, Action, and Regulation

Show how bounded agents choose actions, use tools, plan, receive feedback, and decide whether to continue.

1. Chapter 5 — Choosing Actions, Using Tools, and Coupling to the Environment

Explain how an agent chooses among thinking, asking, acting, and stopping, and how tool contracts constrain grounded behavior.

2. Chapter 6 — Planning, Decomposition, and Replanning

Present planning as a selective control strategy over an action space, not as a synonym for intelligence.

3. Chapter 7 — Grounding, Feedback, Reflection, and Structural Error Analysis

Show how evidence-bearing checks, feedback, and redesignable loops correct structural failures.

4. Chapter 8 — Stopping, Clarification, Handoff, and Bounded Autonomy

Complete the control story by defining principled endings, clarification, escalation, and autonomy limits.

Part III — Architectural Judgment and Synthesis

Turn component knowledge into comparison, task-architecture fit, scope boundaries, and a defended capstone design.

1. Chapter 9 — Comparing Architectures by Task Shape and Scope

Compare candidate architectures by horizon, uncertainty, memory need, tool dependence, feedback richness, and autonomy bound.

2. Chapter 10 — Designing the Literature-Review Assistant

Synthesize a bounded literature-review assistant from scratch, justify every component, and defend the final design with a trace and revision.

Assumptions and notation

Assumptions

- The reader is comfortable with abstraction, boxes-and-arrows reasoning, and tracing stepwise processes. Basic programming literacy is helpful but not required.
- Reinforcement learning as policy learning, retrieval internals, symbolic planning languages and solvers, deployment, observability, governance, framework tutorials, and implementation-heavy code walkthroughs are outside this book's scope.
- The literature-review assistant is treated as a bounded information-centric system with conceptual search, reading, note, and citation-check interfaces, not as a production product or autonomous research platform.

Notation used across the book

Symbol	Meaning
$G = \langle \text{objective, constraints, success criteria, stop/handoff conditions} \rangle$	Task specification bundle.
t	Step index in an execution trace.
C_t	Current model context at step t .
S_t	Explicit external or control state carried across steps.
O_t	Observation or result obtained at step t .
P	Current plan structure.
M	Persistent memory stores and their policies.
$A_t \in \{\text{think, ask, act, stop}\}$	Compact notation for next-step choice when useful.

Part I — Architectural Foundations

Establish the difference between a model and an agent, then build the representational backbone of goals, state, context, and memory.

Chapter 1 — From Generative Model to Agentic System

AA-S01 — From model to agent

Design pressure from the running case

A reader asks for a bounded literature review:

Running request used across the book

Prepare a 1,200-word literature review on citation-grounded question-answering assistants for students, limited to 2021-2024. Use 8-12 papers if available. Separate architecture patterns from evaluation claims. Include a short comparison matrix, and stop or ask if the scope is ambiguous or the evidence is too thin.

A powerful model can often draft something that looks like an answer immediately. A fixed workflow can also run a sequence such as search → read → summarize. Both are useful. But neither, by itself, guarantees the property that matters in this book: **goal-directed behavior over time under explicit limits**.

A prompt-only response may sound polished while silently dropping the request's stopping rule, citation expectations, or scope boundaries. A scripted workflow may preserve a sequence of steps yet still fail when the first search returns poor evidence and the system should adapt. The design pressure in this chapter is therefore simple: **what has to be added around the model before it is sensible to call the whole system agentic?**

Chapter overview

This chapter establishes the book's central lens. The model is the generative component. The agentic system is the organized arrangement around that model: task representation, loop, state, observation handling, action choice, and stopping logic. The goal is not to deny the model's capabilities. It is to locate them precisely, so later architectural additions have a clear reason to exist.

By the end of the chapter, you should be able to read a system description and answer three first questions:

1. Where does the objective live?
2. What persists across steps?
3. What mechanism chooses whether to continue, ask, act, or stop?

Core concepts and definitions

A **model** is the generative component that maps current input context to a next output. In this book, it is usually a language model, but the architectural points do not depend on one vendor or product.

An **interactive system** responds turn by turn, usually with the user providing the next move. It may be capable, but it does not necessarily maintain explicit goal-conditioned control across steps.

A **scripted workflow** executes a predetermined sequence of stages. It can be effective when the task shape is stable, but adaptation is limited to what the script already anticipates.

An **agentic system** is a goal-conditioned loop with explicit state updates and adaptive next-step choice influenced by observations. The system does not merely produce text. It decides, within bounds, how to continue.

The minimal control loop in this book has five conceptual elements:

- **observe** what happened or what is available now,
- **think** about what that means for the current task,
- **act** when an external step is justified,
- **update** state or memory so later steps inherit the right information,
- **stop** or hand off when continuing is no longer justified.

A system can be **more or less agentic**. Agenticness is not a magic binary. It increases when more of the following are explicit and operational rather than implicit and fragile: goal representation, cross-step state, observation-driven adaptation, bounded autonomy, and stop or handoff logic.

Nearest confusable distinction — model vs agentic system

The model is the generator. The agentic system is the organized whole around it. Saying “the model decided to search again” is usually sloppy analysis. Better: “the system used the model inside a loop that selected another search step after receiving an observation.” The point is not verbal purity. It is causal clarity.

Intuition first

A generative model is excellent at **local coherence**. Give it the current context, and it can often continue that context in a useful way. But a multi-step task is not just a long continuation. It is a sequence of choices under constraints, with changing evidence, partial progress, and reasons to stop.

The running literature-review request makes that distinction visible. A one-shot answer can produce a plausible review shape because the model has learned many patterns of academic writing (Brown et al., 2020). Yet the request is not only “write a review.” It also says:

- stay within a date range,
- use a bounded number of papers,
- separate architecture from evaluation claims,
- include a comparison matrix,
- stop or ask if evidence is thin or scope is unclear.

Those conditions matter **during** execution, not only at the beginning. The system must keep them live while it searches, screens, reads, summarizes, and decides whether it has enough evidence. That is architectural work.

A useful informal test is this: if the system's future behavior depends on what it observes along the way, and if that dependence is implemented through explicit control rather than a single static prompt, then you are moving from model use toward agent architecture.

Technical breakdown

What the model can do on its own

A strong generative model can:

- interpret instructions in natural language,
- generate candidate decompositions or summaries,
- produce plausible next steps in text,
- imitate genres such as reviews, notes, or plans,
- compress and restate observed material.

These are important capabilities. They are the reason agentic systems built around generative models are worth designing at all.

What the model cannot sustain by itself

A bare model call does **not** automatically provide:

- durable explicit state across steps,
- guaranteed preservation of success criteria,
- a persistent record of what has already been tried,
- policy for what to write to memory and when,
- principled action choice among think, ask, act, and stop,
- explicit stop or handoff conditions.

Even when a long prompt appears to “contain everything,” it is still only the current input context. If later steps depend on information not reliably kept or updated, the task drifts.

Three system categories on the same bounded request

Table 1.1 compares three ways to handle the running literature-review request.

System form	Where the goal lives	What persists across steps	How next steps are chosen	Typical failure on the running case
Prompt-only interactive response	Mostly in one instruction block and the user's memory	Little beyond the current turn	The user drives continuation	Looks complete too early; cannot independently gather evidence or stop for principled reasons
Fixed scripted workflow	In the script designer's sequence	Whatever fields the workflow explicitly passes along	Predetermined by the script	Keeps going even when search results show the initial

System form	Where the goal lives	What persists across steps	How next steps are chosen	Typical failure on the running case
				decomposition is wrong
Minimal adaptive loop	In explicit task specification plus loop state	External state that is updated after each observation	Chosen at runtime from current observations	Can still loop badly if stop rules are weak or state is poorly placed

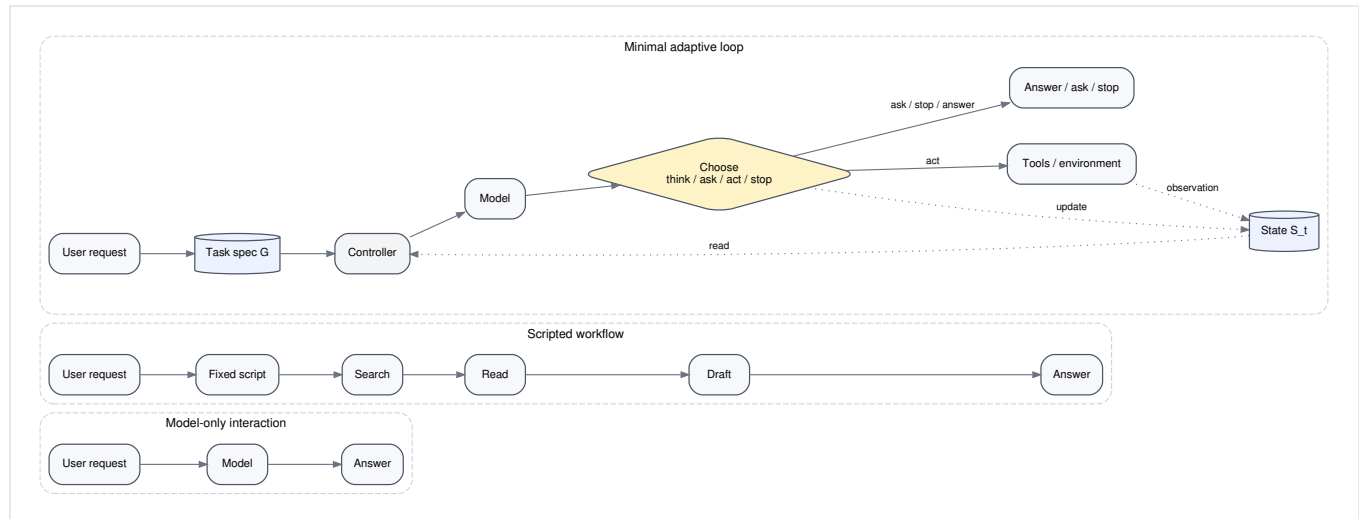


Figure 1.1. The same bounded literature-review request handled as model-only interaction, scripted workflow, and minimal loop-based system. Only the third explicitly ties future control to observations and state updates.

The point is not that prompt-only or scripted systems are useless. They are often exactly right for small, stable tasks. The point is that **agenticness is a property of control structure**, not a synonym for impressive model output.

Minimal loop anatomy

The smallest useful loop can be expressed with light notation:

- Task specification: G
- Current context for the next model call: C_t
- Explicit carried state: S_t
- New observation: O_t
- Chosen next move: $A_t \in \{\text{think}, \text{ask}, \text{act}, \text{stop}\}$

A conceptual step is:

1. Build C_t from G , relevant parts of S_t , and the newest observation.
2. Use the model to propose or score the next move.
3. Execute the chosen move.
4. Record the result in $S_{(t+1)}$.
5. Evaluate whether to continue.

That loop is architectural because the important work is not “a better prompt.” It is the placement of task information, the update path for state, and the control decision about what happens next.

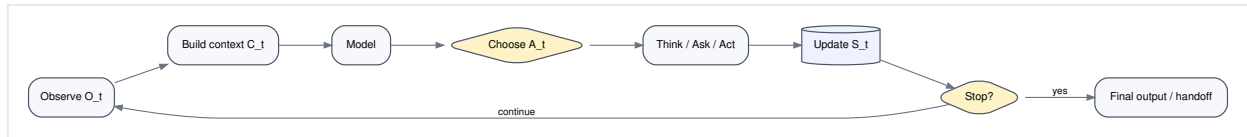


Figure 1.2. Minimal loop grammar used throughout the book. The model participates inside the loop; it is not the loop itself.

Why tool use is not enough

A recurring misconception says: if a model can call tools, it is an agent. Not necessarily. Tool use expands what the system can know and do (Schick et al., 2023), but a script can also call tools. The architectural question is still: **who decides when to call the tool, based on what observations, under what state and stopping rules?**

A model that always issues the same search call inside a fixed pipeline is still closer to a scripted workflow than to an adaptive agentic system. Tool access changes capability. It does not, by itself, create goal-directed control.

Degrees of agenticness

It is better to think in graded terms. A system becomes more agentic when it has more of the following:

1. **Explicit task representation.** The objective, constraints, and stopping conditions are operational.
2. **Cross-step state.** The system preserves the information needed for later choices.
3. **Observation-driven control.** New evidence changes what happens next.
4. **Adaptive action choice.** The next move is not fixed in advance.
5. **Bounded autonomy.** The system may continue acting within limits without waiting for a user every turn.
6. **Stop or handoff logic.** The system can decide not only how to continue, but when not to.

Later chapters will expand each dimension. For now, the main lesson is that agency is primarily **architectural scaffolding around local generation** (Xi et al., 2023).

Worked comparison on the running case

Suppose the system receives the bounded review request.

Prompt-only response. The model writes a clean literature brief immediately. It may mention familiar papers and even structure the answer into themes. But if asked, “Which claims were grounded in read sources rather than prior model knowledge?” the architecture has no explicit answer. The state of evidence was never externalized.

Fixed script. The workflow executes: issue two search queries, read the top eight results, summarize each, then draft the review. This is often better because at least it touches evidence. Yet if the first two queries mostly retrieve tutoring chatbots without citation support, the script still proceeds. It cannot decide that the query family is wrong or that the evidence is too thin.

Minimal adaptive loop. The loop receives the same request, searches once, observes that results mix general tutoring agents with a smaller subset on citation-grounded answering, and chooses to refine the query rather than summarize immediately. That is the smallest architectural move that begins to deserve the term *agentic*: the system changed course because an observation altered control.

Pseudocode example

```
procedure MINIMAL_AGENT_RUN(user_request):
  G <- compile_bounded_task_spec(user_request)
  S0 <- initialize_state(G)

  for t = 0, 1, 2, ...:
    O_t <- latest_observation(S_t)
    C_t <- build_context(G, S_t, O_t)
    A_t <- choose_next_move(C_t)      # think | ask | act | stop

    if A_t == think:
      S_(t+1) <- update_state_with_reasoning(S_t)
    else if A_t == ask:
      return request_clarification(G, S_t)
    else if A_t == act:
      R_t <- execute_external_step(S_t)
      S_(t+1) <- update_state_with_result(S_t, R_t)
    else if A_t == stop:
      return produce_or_handoff_result(S_t)

    if stop_condition_met(G, S_(t+1)):
      return produce_or_handoff_result(S_(t+1))
```

This pseudocode is deliberately sparse. It hides implementation detail and foregrounds structure: explicit task specification, state, observations, adaptive next-step choice, and a stop path.

Cumulative artifact update

After this chapter, the literature-review assistant has gained its first visible architecture artifact:

- a **minimal adaptive loop**,
- a named control vocabulary: observe / think / act / update / stop,
- the analytic distinction between **model**, **scripted workflow**, and **agentic system**.

That loop is still too weak to run well. It has no explicit task specification, no stable state schema, no memory policy, and no stop discipline. Those are the next chapters.

What breaks if this component is missing or misused?

If the loop is missing, the literature-review assistant becomes one-shot or user-driven. It may produce elegant text but cannot independently gather, compare, or revise evidence.

If the loop is present but misused, two common failures appear:

1. **Pseudo-agency.** The system calls itself an agent, but the next step is effectively predetermined. The loop is decorative.
2. **Loop churn.** The system can continue but lacks task-specific reasons to stop, so it keeps searching or rewriting without improving the result.

The lesson is already visible: adding a loop solves one class of failure while introducing another. Architectural design is always about trade-offs, not magic upgrades.

Failure modes, limitations, and misconceptions

- **“An agent is just a chatbot with a better prompt.”** No. A better prompt can improve a model call, but agenticness depends on control structure, state, and adaptive continuation.
- **“Any tool use makes a system agentic.”** No. Tools enlarge the action space. A script can use tools without adaptive control.
- **Anthropomorphizing the model.** Saying “the model wanted to look again” hides the real mechanism. The architecture selected another step because of state and observation handling.
- **“You need reinforcement learning or formal planning before this topic makes sense.”** No. Those are neighboring areas, not prerequisites. This book starts with bounded control loops, not learned policies or formal planners (Sutton and Barto, 2018; Russell and Norvig, 2021).

Chapter summary

A generative model is the engine of local generation. An agentic system is the organized structure that turns that engine into bounded goal-directed behavior over time. The minimal ingredients are an explicit task representation, a loop, state updates, observation handling, adaptive next-step choice, and a way to stop.

This chapter provides the book's first stable distinction: **model $\neq$ system**. The next chapter asks what exactly the system is trying to do, and how to represent that task clearly enough to steer the loop.

References and further reading

- (Brown et al., 2020) for context-bounded generation as the substrate beneath later architecture.
- (Yao et al., 2022; Schick et al., 2023) for examples that interleave generation with action.
- (Xi et al., 2023) for a broad survey of large-language-model-based agent research.
- (Russell and Norvig, 2021; Sutton and Barto, 2018) for neighboring traditions that this book will reference but not require.

Architecture delta

- **Added:** a minimal goal-conditioned loop around the model.

- **Failure addressed:** one-shot or purely scripted behavior that cannot adapt to observations.
- **New risk introduced:** looping without clear task representation or stop discipline.

Chapter 2 — Goals, Task Specifications, and Success Criteria

AA-S02 — Goals, task specification, and success criteria

Design pressure from the running case

The loop from Chapter 1 can continue. But continue **toward what**, exactly?

The literature-review assistant's request sounds clear until the first branching question arrives:

- Does “for students” mean K-12 tutoring, university study support, or library search assistants?
- Are preprints acceptable, or only peer-reviewed papers?
- Does “citation-grounded” mean inline citations, quote verification, or any retrieved evidence?
- When has the assistant done enough work to stop?

A loop without a clear task specification is like steering with a wheel that is not connected to the tires. The system can move, but the motion is not answerable to a defined objective. This chapter therefore turns vague intent into an **operational design object**.

Chapter overview

This chapter distinguishes the user's request from the system's task specification. It introduces the book's lightweight notation

`G = <objective, constraints, success criteria, stop/handoff conditions>`

and uses it to make control possible. The point is not bureaucratic documentation. It is that goals only guide behavior when they are represented clearly enough to affect what the system does next.

By the end of the chapter, you should be able to explain why two systems given the “same request” may require different loops, memory policies, and stop logic if their task specifications differ.

Core concepts and definitions

A **user request** is what the person asks for in natural language.

A **task specification** is the full operational bundle the system uses for control: objective, constraints, success criteria, and stop or handoff conditions.

A **goal** is the objective component inside that bundle.

A **subgoal** is a temporary objective introduced in service of the larger task.

Success criteria define what counts as adequate completion. They are not the same thing as the goal. “Produce a literature review” is a goal. “Every substantive claim is

supported by at least one read source, and the review contains a comparison matrix” are success criteria.

Stop criteria define when the system should stop trying to continue. **Handoff conditions** define when it should escalate to a user or another agent rather than continue autonomously.

A task can be **open-ended** or **bounded**. Open-ended tasks invite indefinite expansion. Bounded tasks specify limits on scope, evidence, depth, or autonomy.

Nearest confusable distinction — user request vs task specification

A user request is what the person says. A task specification is what the system actually runs on. Good systems do not assume those are identical. They transform the request into something operational.

Likewise, **goal** is only one part of the task specification. Constraints, success criteria, and stop or handoff logic are separate architectural commitments.

Intuition first

Many failures attributed to “weak reasoning” are actually failures of task specification.

Suppose the assistant hears: “Review recent work on citation-grounded question-answering assistants for students.” If it simply starts searching, several hidden choices have already been made without discussion: what “recent” means, whether educational chatbots without citations count, what the deliverable looks like, how much evidence is enough, and when ambiguity is serious enough to ask a question.

That hiddenness matters because the architecture cannot be judged if the task is not explicit. A system that returns a polished three-paragraph overview may look successful until the user expected a comparison matrix with 10 sources. Another system may appear slow or fussy because it asks for clarification, yet that clarification may be exactly what keeps the architecture bounded and correct.

In human-computer interaction, clarification is not a failure state. It is a design move for aligning system behavior with user intent (Amershi et al., 2019). In this book, clarification is also a control action: sometimes the right next step is **ask**, not **act**.

Technical breakdown

The task-spec tuple

We will use this light notation throughout the book:

$G = \langle \text{objective, constraints, success criteria, stop/handoff conditions} \rangle$

For the running case, a stronger task specification than the raw request is:

- **objective:** produce a 1,200-word literature review on citation-grounded question-answering assistants for students;

- **constraints:** use material from 2021–2024, include 8–12 papers if available, separate architecture patterns from evaluation claims, include a short comparison matrix, avoid implementation detail and vendor-specific products;
- **success criteria:** every substantive claim is grounded in at least one read source, the review covers at least three architecture patterns, the comparison matrix includes source, grounding method, tool use, and limitation fields, and uncertainty is surfaced where evidence is thin;
- **stop/handoff conditions:** ask if “for students” or “citation-grounded” is materially ambiguous, stop when the success criteria are met with adequate evidence, and hand off if accessible evidence is too thin or conflicting in ways the assistant cannot resolve responsibly.

This is longer than the user request. That is fine. Control needs more structure than conversation often provides.

Request-to-task-spec transformation

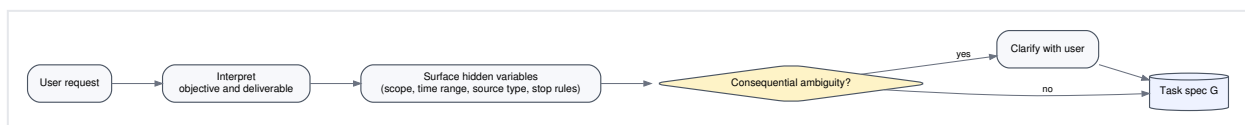


Figure 2.1. The user request becomes an operational task specification through interpretation, clarification when needed, and explicit bounding decisions.

The transformation usually involves four moves:

1. **Interpret the request.** Identify the apparent objective and deliverable shape.
2. **Surface hidden variables.** Time range, source type, depth, domain, risk, and autonomy limits.
3. **Clarify when ambiguity is consequential.** Not every ambiguity matters enough to ask about; the important question is whether it changes architecture or evaluation.
4. **Compile the bounded task specification.** Make the objective, constraints, success criteria, and stop/handoff logic explicit.

Deliverable shape is part of control

The architecture behaves differently depending on what the system is trying to deliver.

A short bullet summary of three papers may need only a short horizon and little memory. A literature review with a comparison matrix, uncertainty notes, and evidence requirements needs stronger state handling, maybe planning, and certainly explicit stop logic.

This is why “do the task well” is not a usable objective. It leaves the system with no criterion for whether to search more, summarize now, or ask the user to narrow scope.

Open-ended vs bounded formulations

Compare the following two task formulations.

Formulation	What it tells the system	Architectural consequence
“Review recent work on educational QA assistants.”	Broad topic, no limits, no success threshold, no stop condition	Encourages open-ended search and makes stopping arbitrary
“Produce a 1,200-word review of citation-grounded QA assistants for students from 2021–2024, using 8–12 papers if available, with a comparison matrix and explicit uncertainty notes.”	Scope, horizon, deliverable, evidence target, and completion cues	Supports bounded search, controlled note-taking, and principled stop decisions

A bounded formulation is not always better for the person. Some users truly want open exploration. But the architecture must know which case it is in. Open-ended tasks generally need more user re-entry, stronger clarification, and more explicit autonomy limits.

Subgoals are not the final goal

Subgoals help the system make progress, but they must remain subordinate to the main task. For the running case, candidate subgoals are:

1. generate initial search terms,
2. retrieve candidate sources,
3. screen sources for relevance,
4. capture notes,
5. synthesize architecture patterns,
6. verify claims and citations,
7. stop or hand off.

None of these is the final goal. A system that treats “retrieve papers” as success has confused a subgoal with the task itself. This confusion is common in agent design because intermediate steps are visible and satisfying. But an executed search is not a completed literature review.

Clarification as legitimate architecture

Clarification becomes the right move when ambiguity changes one of the following:

- the source universe to be searched,
- the risk or reversibility of action,
- the deliverable shape,
- the success threshold,
- the stop or handoff logic.

If “for students” could mean either tutoring systems or scholarly search assistants, the query set, screening rules, and evaluation dimensions will differ. In that case, clarification is justified. If the ambiguity is minor and does not change control, the system can proceed while noting its assumption.

A good clarification question is narrow, decision-relevant, and easy to answer. For example:

Do you mean assistants used by students for studying and tutoring, or systems that help students search and read literature? The scope changes which papers count as relevant.

That question is not a conversational nicety. It is an architectural act that sharpens G.

Worked example: from vague request to bounded task specification

Suppose the raw request is:

“Please review recent work on citation-grounded question-answering assistants for students.”

A bounded compilation could be:

```
G_review = <objective = produce a 1,200-word literature review; constraints = 2021–2024, 8–12 papers if available, architecture-first framing, comparison matrix required; success criteria = each substantive claim grounded in at least one read paper, at least three architecture patterns compared, uncertainty noted where evidence is thin; stop/handoff = ask if scope remains ambiguous, stop when criteria are met, hand off if fewer than five solid sources are accessible or if evidence conflicts on high-stakes recommendations>.
```

Notice two things.

First, some items were **made explicit**, not invented. A good architecture often needs to make hidden expectations operational.

Second, this compiled specification already implies later architectural choices:

- The evidence requirement implies explicit state and maybe memory.
- The comparison matrix implies structured note capture.
- The ambiguity rule implies an available **ask-user** action.
- The stop rule implies the loop must evaluate sufficiency, not just completion of steps.

Pseudocode example

```
procedure COMPILER_TASK_SPEC(request):
  objective <- identify_primary_deliverable(request)
  constraints <- extract_or_assume_bounds(request)
  success <- define_what_counts_as_adequate(objective, constraints)
  stop_handoff <- define_when_to_stop_ask_or_escalate(objective, constraints)

  if consequential_ambiguity(objective, constraints, success):
    return ASK_USER_FOR_CLARIFICATION
  else:
    return <objective, constraints, success, stop_handoff>
```

This pseudocode is intentionally modest. It highlights a key point: task specification is a control artifact that can itself trigger clarification.

Cumulative artifact update

The literature-review assistant now gains its first formal artifact:

Artifact	Current content
Task specification <code>G_review</code>	Objective, constraints, success criteria, and stop/handoff conditions for the bounded literature-review request

This artifact will remain visible across the rest of the book. Later chapters will show how `G_review` influences state placement, memory policy, tool choice, planning, and stopping.

What breaks if this component is missing or misused?

If the task specification is missing, the system tends to fail in one of four ways:

1. **Scope drift.** It keeps widening the topic because nothing in the control state says where the boundary is.
2. **Premature completion.** It stops after producing plausible prose rather than meeting explicit success criteria.
3. **Subgoal capture.** It mistakes a successful search or summary for task success.
4. **Aimless autonomy.** It continues acting without clarity about what would justify stopping or asking the user.

If the task specification is misused, a different problem appears: **overconstraining the system**. A brittle specification can hard-code an unnecessary sequence rather than stating the real objective and limits. The architecture needs guidance, not a script masquerading as a goal.

Failure modes, limitations, and misconceptions

- **“Do the task well” is enough.** It is not. The system needs success criteria it can test against.
- **Collapsing subgoals into the final goal.** Retrieved papers, notes, and summaries are means, not the end.
- **Ignoring stop criteria.** Without explicit stopping logic, loops tend to continue until some external timeout or arbitrary decision.
- **Treating clarification as failure.** Clarification is often the correct next move when ambiguity changes control (Amershi et al., 2019).
- **Assuming more detail is always better.** A task specification should be sufficiently operational, not unnecessarily procedural.

Chapter summary

Goals matter only when they are represented clearly enough to alter control. The user's request is conversational input. The task specification is the architecture's operational target. By expressing the running case as

`G = <objective, constraints, success criteria, stop/handoff conditions>`

we make later design questions concrete: what state must be carried, what memory is needed, when the system should ask, and what counts as enough evidence to stop.

The next chapter asks where that information lives over time. Once the task is explicit, we can trace how an execution unfolds and how state placement affects failure.

References and further reading

- (Amershi et al., 2019) for clarification, expectation management, and user-facing constraints.
- (Parasuraman et al., 2000) for a classic view of choosing levels of automation rather than assuming “more autonomy” is always better.
- (Xi et al., 2023) for examples of contemporary agent systems that implicitly or explicitly compile tasks into control structures.

Architecture delta

- **Added:** an explicit task specification `G_review`.
- **Failure addressed:** vague looping toward an underdefined objective.
- **New risk introduced:** overspecifying the task so rigidly that the “goal” becomes a hidden script.

Chapter 3 — Loops, State, Context, and Execution Traces

AA-S03 — Control loops, state, and context

Design pressure from the running case

The literature-review assistant now has `G_review`, but a new failure appears during execution.

At step 1 it retrieves a mixed set of papers: some are truly about citation-grounded answering; others are generic tutoring agents. At step 2 it screens results and decides to exclude systems that merely retrieve passages without citing them in the final answer. At step 3 it opens a source and takes notes. At step 4 it drafts a synthesis.

And then the exclusion rule disappears.

Why? Because it lived only inside the context of one earlier model call. Nothing in the architecture placed it in a durable external structure. The system behaved as if “context” were the same thing as “state.” It is not.

Chapter overview

This chapter makes execution over time visible. It distinguishes current context from carried external state, world state, and observations, and it uses trace tables as the book's primary explanatory tool. It also revisits a core model limitation from Chapter 1: context-bounded generation is not durable control.

By the end of the chapter, you should be able to inspect a multi-step run and identify where information is located, where it should have been located, and how a failure arose from omission or misplacement.

Core concepts and definitions

Context (`Ct`) is what is present in the current model call at step `t`.

External state (`St`) is explicit information the system carries across steps outside the current context window: variables, task fields, checklists, counters, retrieved item lists, partial products, and control markers.

World state is the relevant state of the external environment: which documents exist, whether a paper can be opened, whether a tool times out, whether the user has responded, and so on.

Observation (`Ot`) is what the system learns from the world or from a tool result at step `t`.

An **execution trace** is what actually happened over time.

A **plan** is intended future structure. A **trace** is realized past behavior.

Nearest confusable distinction — context vs external state vs memory

Context is what the model sees right now. **External state** is what the control loop explicitly carries across steps. **Memory**, introduced in the next chapter, is designed persistence with write and retrieval policies. Do not collapse these into one vague idea of “stuff the system knows.”

Intuition first

A multi-step agent run is not one giant thought. It is a sequence of state transformations.

That matters because a generative model only receives the context assembled for the current step. Even if a system can pass large amounts of text into a model call, that still does not solve the architectural question of **what deserves to persist, where it lives, and how later steps recover it**. Pushing more into the context window is often a temporary workaround, not a durable design (Brown et al., 2020; Packer et al., 2023).

The literature-review assistant makes this visible. Some information belongs in current context because it is immediately relevant to the next choice. Other information belongs in explicit state because later steps depend on it even if it is not worth repeating verbatim every turn. Still other facts belong to the outside world and must be re-observed rather than assumed.

Technical breakdown

The execution cycle

A bounded run usually follows this abstract rhythm:

1. inspect current goal state and relevant world state,
2. assemble the next model context `Ct`,
3. choose a move,
4. execute the move,
5. record what happened in `S(t+1)`,
6. evaluate whether to continue.

The quality of the run depends heavily on **state placement**. If an item matters for later control but is never written into explicit state, the architecture is relying on luck.

Four information locations

For the running case, it helps to classify information by location.

Information example	Best location	Why
The date range 2021–2024	External state <code>S_t</code> and often copied into <code>C_t</code>	It should persist across the whole run
The currently opened paper's abstract	Observation <code>O_t</code> , then maybe state if retained	It arrives from a tool and may or may not deserve persistence
The list of already-screened papers	External state <code>S_t</code>	Later steps need to avoid duplication
Whether a source URL is currently reachable	World state / observation	It may change and should be checked, not assumed

Information example	Best location	Why
The temporary reasoning for whether to read paper 7 next	Current context <code>C_t</code>	It may matter only for the current choice

The point is not to memorize categories. The point is to ask, for each piece of information: **will later control depend on this, and if so where should it live?**

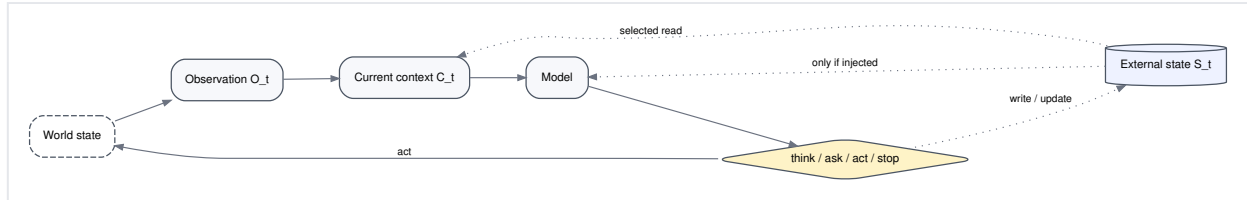


Figure 3.1. Information locations in a bounded loop. Context, external state, world state, and observations occupy different places and should not be collapsed into one bucket of “memory.”

Trace tables

The standard trace table format used in this book is:

t	goal view	context	external state	observation	action	result	stop decision
---	-----------	---------	----------------	-------------	--------	--------	---------------

This is not paperwork. It is a diagnostic instrument.

Table 3.1 shows a simplified trace for the literature-review assistant.

t	goal view	context	external state	observation	action	result	stop decision
0	Need bounded review with matrix	<code>G_review</code> , no sources yet	query list empty; screened set empty	none	think	draft initial queries and screening rules	continue
1	Need candidate sources	initial queries + screening rules	queries stored	search returns 18 mixed results	act	store candidates; flag broad result mix	continue
2	Need relevant citation-grounded systems	top candidates + rules	screened set empty	abstracts show many generic tutoring systems	think	refine exclusion rule: require answer-level citation or quote support	continue
3	Need representative papers	refined rule + candidate titles	screened set updated	7 papers meet rule, 4 uncertain	act	open first relevant paper and capture note	continue
4	Need pattern synthesis	notes so far + open paper result	notes store has 1 note; uncertain set tracked	first paper emphasizes verified quotes	think	add “evidence presentation”	continue

t	goal view	context	external state	observation	action	result	stop decision
						dimension to matrix	

Now compare that with a flawed trace.

t	goal view	context	external state	observation	action	result	stop decision
2	Need relevant citation-grounded systems	top candidates + rules	screened set empty	abstracts show many generic tutoring systems	think	refine exclusion rule	continue
3	Need representative papers	candidate titles only	rule never written to S_t	open a generic RAG paper	act	note taken on irrelevant source	continue
4	Need pattern synthesis	current note only	irrelevant note now treated as valid evidence	none	think	synthesize too broad a claim about “citation-grounded assistants”	continue

The flaw is structural, not mysterious: a criterion needed for future control existed only in transient context.

Why state placement matters

State placement affects at least five kinds of failure:

1. **Omitted state.** Important information is never stored.
2. **Misplaced state.** Information is stored somewhere that later control cannot reliably access.
3. **Overloaded context.** Too much low-value material is kept in every call, crowding out what matters now.
4. **Unobserved world changes.** The system assumes the world stayed the same after an action.
5. **Plan-trace confusion.** The architecture treats intended future steps as if they already happened.

The running case is rich enough to produce all five. A note that “paper 4 looked relevant” is not the same as storing that paper 4 was actually read. A planned comparison matrix is not the same as an existing matrix in state. A retrieved URL is not the same as a reachable document.

Context windows revisited

Chapter 1 introduced the idea of context-bounded generation. Here the consequence becomes operational.

Even with long context windows, context remains a **constructed view** for the current call. It is not the same as durable state. Durable state requires the architecture to decide:

- what to carry forward,
- in what representation,
- with what update rules,
- and how to distinguish current view from persistent facts.

This distinction becomes even sharper once memory enters the picture. Memory will extend persistence. It will not erase the difference between `C_t` and `S_t`.

Plan versus trace

For beginners, one of the most important architectural distinctions is this:

- a **plan** says what the system intends to do,
- a **trace** says what the system actually did.

The literature-review assistant may have a plan like:

1. search broadly,
2. refine scope,
3. read eight papers,
4. synthesize patterns,
5. verify citations,
6. stop.

But the trace may show something different:

1. broad search,
2. broad search again,
3. ask user for clarification,
4. read four papers,
5. discover thin evidence,
6. hand off.

A well-designed architecture treats both objects separately. Otherwise the system may incorrectly assume completion because the plan contained a step that the trace never executed.

Worked example: tracing a bounded run

Let us trace a slightly longer segment of the running case.

- **Step 0:** the system loads `G_review` and initializes state with empty candidate, screened, note, and matrix fields.
- **Step 1:** it issues a search query. The search tool returns mixed results, including papers on general retrieval-augmented generation and papers on explicit answer citation.

- **Step 2:** the system tightens its screening criterion. This criterion is written into external state.
- **Step 3:** it screens abstracts and creates two sets: accepted and uncertain.
- **Step 4:** it opens an accepted paper and captures a structured note: architecture pattern, grounding method, tool dependence, evidence type, limitation.
- **Step 5:** it checks the comparison matrix draft and notices that “user-facing evidence display” has emerged as a dimension. The matrix schema is updated in state.
- **Step 6:** it stops reading new papers temporarily and synthesizes a provisional pattern summary.

Everything above depends on explicit state. Without stored candidate sets, screening rules, note objects, and matrix schema, the next step would be forced to reconstruct its world from partial context.

Pseudocode example

```

procedure EXECUTE_STEP(G, S_t):
  O_t <- observe_world_or_tool_result(S_t)
  C_t <- assemble_context(G, S_t, O_t)

  A_t <- choose_next_move(C_t)

  if A_t == act:
    result <- perform_action(S_t)
    S_(t+1) <- write_result_to_state(S_t, result)
  else if A_t == think:
    update <- derive_control_update(C_t)
    S_(t+1) <- merge(S_t, update)
  else if A_t == ask:
    return clarification_request(C_t)
  else:
    return stop_or_handoff(S_t)

  return S_(t+1)

```

The main design question hidden inside this pseudocode is not the model call. It is the write path `write_result_to_state`.

Cumulative artifact update

The literature-review assistant now gains a second visible artifact: an explicit **trace schema** for its run.

Artifact	Current content	Trace schema	t	goal view	context
external state	observation	action	result	stop decision	State inventory queries, candidate list, screened set, uncertain set, notes, matrix schema, completion markers

This state inventory will become the basis for the memory choices in Chapter 4.

What breaks if this component is missing or misused?

If explicit state is missing, the architecture behaves as though the current context were enough. In the running case, that causes forgotten screening rules, repeated reading of the same papers, and unsupported synthesis.

If state is misused, two opposite errors appear:

- **too little state:** the system loses criteria and results;
- **too much state:** every detail is carried forward, burying what matters for the next choice.

State design is therefore selective. Good architectures preserve what later control needs, not everything the system has ever seen.

Failure modes, limitations, and misconceptions

- **“Context window equals memory.”** No. Context is the current view. It can include retrieved or stored material, but it is not itself a persistence policy.
- **Equating context with all state.** Some information should remain external and structured even if only a subset is injected into the next call.
- **Forgetting world state.** Tool outputs can fail, documents can disappear, and users can change constraints. External reality is not frozen.
- **Confusing plan with trace.** A checklist is not evidence that the steps occurred.

Chapter summary

This chapter made time visible. A running agentic system is a sequence of state transformations, not a single long thought. Context, external state, world state, and observations play different roles. Trace tables reveal whether those roles are being respected. They also prepare the way for the next step: if state must persist beyond the immediate run or across sessions, we need a memory architecture rather than ad hoc carry-forward fields.

References and further reading

- (Brown et al., 2020) for the context-bounded generative substrate.
- (Packer et al., 2023) for a systems-oriented discussion of context limits and memory tiers.
- (Park et al., 2023) for a memorable example where observation, planning, and reflection depend on explicit state structures.

Architecture delta

- **Added:** an explicit distinction among `C_t`, `S_t`, world state, and `O_t`, plus a standard trace table.
- **Failure addressed:** losing crucial criteria or results because they existed only in transient context.
- **New risk introduced:** carrying too much state forward and obscuring what matters now.

Chapter 4 — Memory Types, Persistence, and Policy

AA-S04 — Memory architectures and persistence policies

Design pressure from the running case

After Chapter 3, the literature-review assistant can carry explicit state within a run. But two new problems appear.

In one version, the assistant is almost memoryless. It has good tools, so it repeatedly searches, reopens papers, and reconstructs notes each time a new synthesis question arises. The result is wasteful and fragile.

In the opposite version, the assistant stores everything. Query drafts, rough notes, old summaries from previous topics, user preferences from unrelated sessions, and reusable procedures all land in one undifferentiated store. The result is contamination: stale notes leak into the current review, and the system cannot tell fresh evidence from old convenience.

This chapter addresses that design pressure by treating memory as **representation plus policy**, not simply as “more storage.”

Chapter overview

Memory extends state management across longer horizons. The core questions are:

- what should persist,
- in what form,
- when should it be written,
- when should it be retrieved,
- how should freshness and contamination be handled,
- and when should the system rely on tools instead of memory?

The chapter distinguishes working, episodic, semantic, and procedural or external memory roles and applies them to the bounded literature-review assistant.

Core concepts and definitions

Working memory is short-horizon task material used during the current run: active notes, candidate lists, partial matrix entries, and unresolved questions.

Episodic memory stores records of particular past runs or steps: which queries were tried, what sources were screened out, what clarification the user gave, and what failure occurred.

Semantic memory stores reusable knowledge abstractions: stable concept notes, glossaries, or interpreted relationships that may help future tasks.

Procedural or external memory stores reusable procedures, templates, and structured artifacts outside the model call: review templates, note schemas, citation-check checklists, and other operational patterns.

A **memory policy** specifies write triggers, retrieval triggers, freshness rules, and failure controls.

Nearest confusable distinction — storage vs memory

Not every database or file store is a memory architecture. It becomes memory in the architectural sense only when the system has explicit representation choices and policies for when to write, retrieve, refresh, ignore, or forget.

Likewise, memory is not the same as state. State may be entirely run-local. Memory is designed persistence beyond the immediate step and often beyond the immediate run.

Intuition first

If Chapter 3 asked “where does this information live right now?”, Chapter 4 asks “what deserves to persist and under what rules?”

The running literature-review assistant illustrates why that second question matters. Some information is worth keeping only within the run: current candidate papers, temporary synthesis fragments, and today's uncertainty flags. Some may be useful later: a concept note explaining the difference between retrieved evidence and answer-level citation support, or a review procedure template that can structure future tasks. Other information should probably **not** persist in a reusable way: an early mistaken interpretation of the topic, or a rough note whose claims were never verified.

Memory helps when it reduces repeated work without corrupting future control. Memory hurts when it reintroduces outdated or weakly grounded material simply because it is available.

Technical breakdown

Four roles of memory in the running case

For the literature-review assistant, the candidate memory stores are:

1. **Temporary scratch notes** for the current review.
2. **Query and screening history** from the current or recent runs.
3. **Reusable concept notes** about architecture distinctions.
4. **Procedure templates** for how to conduct a bounded review.

These do not all deserve the same treatment.

Store candidate	Memory role	Why it exists	Should it persist beyond the current run?
Scratch notes on currently read papers	Working memory	Support current synthesis	Usually no, unless distilled
Query history and screened-out list	Episodic memory	Avoid repeated failed searches and duplicate reading	Sometimes, with freshness checks

Store candidate	Memory role	Why it exists	Should it persist beyond the current run?
Stable concept note on “grounding vs citation formatting”	Semantic memory	Reuse a clarified distinction across future reviews	Yes, if curated
Literature-review checklist template	Procedural memory	Reuse a reliable procedure	Yes, if versioned

The important architectural idea is that **facts, notes, plans, and procedures are not the same kind of thing**. Putting them into one store with one retrieval rule is asking for trouble.

Representation matters

Suppose the assistant stores a paper note as free text:

“Interesting paper. Uses retrieval and gives references. Might fit.”

That is weak representation. Later control cannot easily tell:

- what claim was supported,
- whether the references were answer-level or merely background,
- whether the note is verified,
- when it was written,
- or whether the paper was finally included.

A stronger representation is a structured note:

- source identifier,
- topic fit,
- grounding method,
- evidence type,
- limitation,
- verification status,
- date captured.

The book deliberately avoids retrieval internals, embeddings, or indexing algorithms. The architectural point is simpler: **representation determines what later policy can do**.

Memory is policy, not just storage

The standard memory-policy table used in this book is:

store	contents	write trigger	retrieval trigger	freshness rule	failure mode
-------	----------	---------------	-------------------	----------------	--------------

Table 4.1 proposes a minimal policy for the running case.

store	contents	write trigger	retrieval trigger	freshness rule	failure mode
Scratch pad	current notes, draft matrix				

store	contents	write trigger	retrieval trigger	freshness rule	failure mode
	rows, unresolved questions	after each read or synthesis step	next step in same run	clear at task end unless distilled	noisy carryover, clutter
Query history	issued queries, result quality, screened outcomes	after search or screen step	when search seems stalled or repetitive	valid only within topic and time scope	anchoring on bad early queries
Concept notes	curated architecture distinctions and definitions	only after verification or deliberate curation	when the task invokes the same concept	periodic review; do not treat as current evidence	stale abstractions masquerading as evidence
Procedure templates	review checklist, note schema, stop checklist	after stable procedure proves useful	at task initialization	update when policy changes	rigid reuse on tasks with different shape

Notice what the table does **not** say. It does not say “store everything because it might help later.” Good memory design is selective.

Freshness and contamination

Two of the most important memory policies are:

- **freshness:** when is a stored item still trustworthy for the current task?
- **contamination control:** how do we prevent old or irrelevant material from shaping the current run?

For the literature-review assistant, a semantic note about the concept of grounding may stay useful for a long time. A note that “paper X is a representative 2023 system” may age quickly, especially if the current review has a tighter domain or newer scope. Query history from a prior healthcare review is not only stale; it is likely misleading.

A helpful rule is to separate **reusable concepts** from **task evidence**. Reusable concepts may persist with slower refresh. Task evidence must be tightly scoped and time-stamped.

When memory helps, and when tools are better

Memory is not always the right answer. The architecture should prefer fresh observation from tools when:

- the world may have changed,
- evidence must be current,
- the cost of rechecking is low,
- and stale memory would be risky.

In the running case, it is often better to reopen a paper or rerun a query than to trust an old vague note. By contrast, a stable procedure template such as a note schema is exactly the kind of thing memory should preserve.

This trade-off leads to a recurring comparison used throughout the book.

Memory-rich vs tool-rich variants

The bounded literature-review assistant can be made **memory-rich and tool-poor** or **memoryless-tool-rich**. Neither is universally best.

Variant	Strength	Weakness	Typical use
Memoryless-tool-rich	Fresh evidence, less contamination, simple policy	Repeats work, may be slow, weak continuity	Short tasks or highly dynamic evidence
Memory-rich-tool-poor	Reuses structure and past work, stronger continuity	Risk of stale notes, policy complexity, hidden leakage	Recurring task families with curated stores

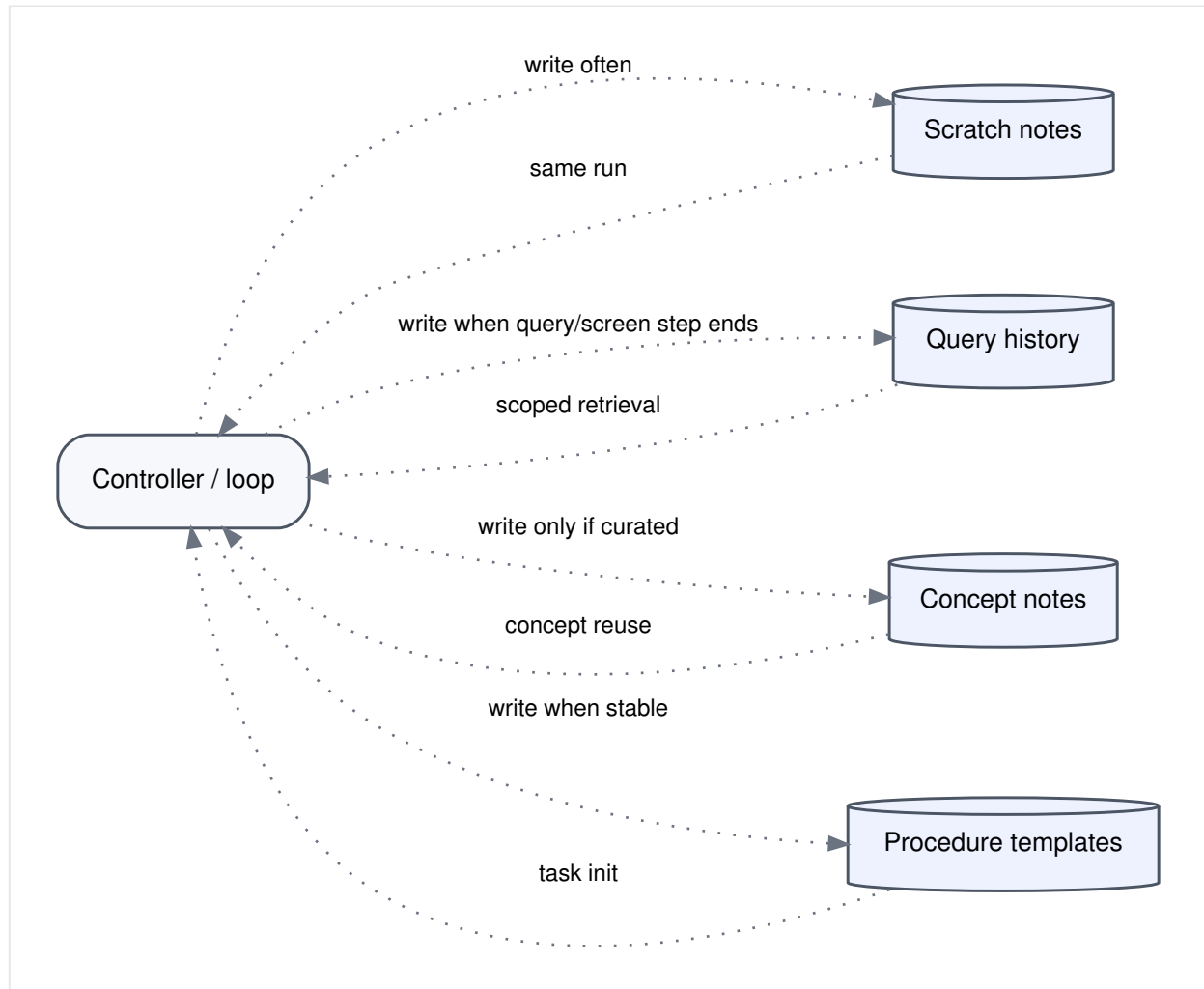


Figure 4.1. Distinct stores with different write and retrieval policies. The dotted arrows show selective reads and writes rather than indiscriminate dumping into one store.

In practice, the strongest design for the running case is a **minimal memory architecture**: curated procedure templates, scoped episodic query history, and run-local scratch notes, with semantic notes allowed only after deliberate curation.

Worked example: classifying candidate stores

Let us classify the running case's candidate stores one by one.

Temporary scratch notes. These belong to working memory. They help current synthesis and should usually be cleared at task end unless condensed into something reusable.

Prior search or query history. This is episodic. It can prevent repetition within a run and sometimes across closely related runs. But it needs topic scope and freshness checks.

Reusable concept notes. These are semantic. They should be rare and curated. A distilled note about the difference between “retrieval” and “answer-level evidence display” can help future reviews. A half-verified claim about one paper should not enter semantic memory.

Review procedure templates. These are procedural. A reliable checklist for screening, note capture, and citation verification is a good candidate for persistence.

Pseudocode example

```
procedure MEMORY_WRITE_DECISION(item, store_type):
  if store_type == scratch:
    write(item)
  else if store_type == episodic:
    write_if(item.is_scoped and item.is_likely_reused_soon)
  else if store_type == semantic:
    write_if(item.is_curated and item.is_verified)
  else if store_type == procedural:
    write_if(item.is_generalizable and item.is_stable)

procedure MEMORY_RETRIEVE(query, current_task):
  candidates <- fetch_relevant_items(query)
  return filter(candidates,
    scope_matches(current_task)
    and freshness_ok(current_task)
    and contamination_risk_low)
```

The important operations are not mathematically fancy. They are policy checks.

Cumulative artifact update

The literature-review assistant now gains a minimal memory policy.

Artifact	Current content
Memory policy	distinct stores for scratch notes, episodic query history, curated concept notes, and procedure templates
Explicit rule	do not mix task evidence, reusable concepts, and procedures in one undifferentiated store

This chapter also introduces the book's second recurring comparison: **memoryless-tool-rich** versus **memory-rich-tool-poor**.

What breaks if this component is missing or misused?

If memory is absent, the system redoes work, forgets useful structure, and cannot build continuity across longer tasks.

If memory is misused, the failures are often worse:

1. **staleness:** old notes are treated as current evidence,
2. **contamination:** unrelated past runs leak into the present task,
3. **preference leakage:** prior user-specific or task-specific assumptions shape a new review,
4. **store collapse:** facts, plans, and procedures become indistinguishable.

For the running case, the most dangerous failure is not forgetting. It is remembering the wrong thing in the wrong form.

Failure modes, limitations, and misconceptions

- **“Every database is memory.”** No. A database becomes memory only when representation and policy are explicit.
- **“More memory is always better.”** No. More poorly governed memory often makes control worse.
- **Treating facts, plans, and scratch notes as one store.** This destroys retrieval precision and policy clarity.
- **Ignoring freshness.** Old notes can be perfectly consistent and still unfit for the current task.
- **Using memory to avoid observation.** When current evidence matters, tools should often win over stored guesses (Lewis et al., 2020; Packer et al., 2023).

Chapter summary

Memory is not “extra context.” It is designed persistence governed by explicit policy. Different stores serve different roles, and the literature-review assistant benefits most from a small number of carefully governed stores rather than indiscriminate accumulation. The next chapter moves from persistence to action: once the system knows what it is trying to do and what it remembers, how does it choose between thinking, asking, acting, and stopping?

References and further reading

- (Lewis et al., 2020) for explicit non-parametric memory in generation.
- (Park et al., 2023) for a memorable architecture that distinguishes observation, memory, reflection, and planning.
- (Packer et al., 2023) for a layered memory view motivated by context limitations.

Architecture delta

- **Added:** distinct memory roles and an explicit memory-policy table.
- **Failure addressed:** repeated work or ad hoc persistence that loses structure.
- **New risk introduced:** stale, contaminated, or overgeneralized memory influencing current control.

Part II — Deliberation, Action, and Regulation

Show how bounded agents choose actions, use tools, plan, receive feedback, and decide whether to continue.

Chapter 5 — Choosing Actions, Using Tools, and Coupling to the Environment

AA-S05 — Action selection, tools, and environment interfaces

Design pressure from the running case

The literature-review assistant now has a task specification, explicit state, and a minimal memory policy. It still faces a crucial decision every step:

- Should it think a little longer?
- Should it ask the user to clarify scope?
- Should it call a search, reading, or citation-check tool?
- Should it stop?

This is the first place where architecture truly meets the world. The book's running case becomes operational only when the system can select among **reasoning**, **asking**, **acting**, and **stopping** under explicit interface assumptions.

Chapter overview

This chapter introduces the `think / ask / act / stop` choice set, tool contracts, environment interfaces, and a primer on grounding. The main lesson is that tools do not merely add capabilities. They change what the system can know, what side effects it can cause, and what counts as evidence.

By the end of the chapter, you should be able to describe a tool not as “something the agent can use,” but as a contract with inputs, preconditions, side effects, returned observations, and limits.

Core concepts and definitions

A **reasoning step** updates interpretation or control without directly changing the external environment.

An **action step** changes the external environment or retrieves information from it through a tool interface.

A **meta-control step** changes the control mode itself, such as asking for clarification, replanning, stopping, or handing off.

The compact choice set used in this book is:

`A_t ∈ {think, ask, act, stop}`

A **tool contract** specifies:

- inputs,
- preconditions,
- side effects,
- observation returned,
- limits.

An output is **grounded** when it is appropriately tied to observations or evidence from tools or the environment. A textual artifact is not grounded merely because it came from a tool.

Nearest confusable distinction — successful tool call vs successful task

A tool call can succeed technically while the task still fails conceptually. A search tool may return results; those results may still be irrelevant. A note-writing tool may save a note; the note may still be unsupported. Tool success and task success are different layers of evaluation.

Intuition first

When beginners first encounter agent systems, tools often look like the exciting part. Search! Browsers! Calculators! File systems! But architecture begins one step earlier. The question is not “what tools are available?” It is “under what conditions is a tool call the right next move?”

The literature-review assistant demonstrates why that matters. If the current ambiguity is about whether “for students” means tutoring systems or literature-search assistants, calling search immediately may be inferior to asking the user. If the current synthesis claim needs support, a citation-check tool may be better than additional free-form reasoning. If enough evidence already exists, further search may be worse than stopping.

A well-designed agent does not default to action. It justifies action.

Technical breakdown

The action-choice frame

At any step, the system chooses among four broad moves:

- **think**: revise interpretation, decide what matters next, or update local control state;
- **ask**: get missing task information from the user;
- **act**: query or affect the environment through tools;
- **stop**: end the run or prepare a handoff.

This framing matters because it prevents a common design mistake: treating all uncertainty as a cue to “think harder.” Sometimes uncertainty should trigger observation, not more generation. Sometimes it should trigger clarification or stopping.

Tool affordances and limits

A tool changes two things at once:

1. what the system can **do**,
2. what the system can **know**.

For the running case, the conceptual tools are:

- **search** for candidate sources,
- **document read** for paper content,

- **note write** for structured note capture,
- **citation check** for validating whether claims are linked to actual sources.

Each tool has affordances and limits. Search can retrieve candidates, but it does not confirm relevance or factual support. Reading can expose source content, but only for accessible documents. Note writing persists an interpretation, but the note's quality depends on what was read and how it was represented. Citation checking may confirm that a cited source exists in the note set, but not necessarily that the source supports the full claim.

Tool contracts for the running case

The standard tool-contract table format is:

```
tool | inputs | preconditions | side effects | observation returned | limits
```

Table 5.1 sketches the literature-review assistant's conceptual tool contracts.

tool	inputs	preconditions	side effects	observation returned	limits
Search	query, time range, source scope	query is within task scope	none outside retrieval logs	ranked candidate list with metadata	result quality depends on query and index coverage
Document read	source identifier or URL	source is reachable and permitted	may consume time budget	abstract, sections, references, quotes	inaccessible or noisy documents may block progress
Note write	source identifier, structured fields	source has been read or observed sufficiently	writes to note store	confirmation of stored note	bad note structure can fossilize weak interpretation
Citation check	draft claim, candidate supporting sources	supporting sources have been read and indexed in state	may flag claim as unsupported	support verdict, mismatch, or ambiguity	only as good as source coverage and mapping rule

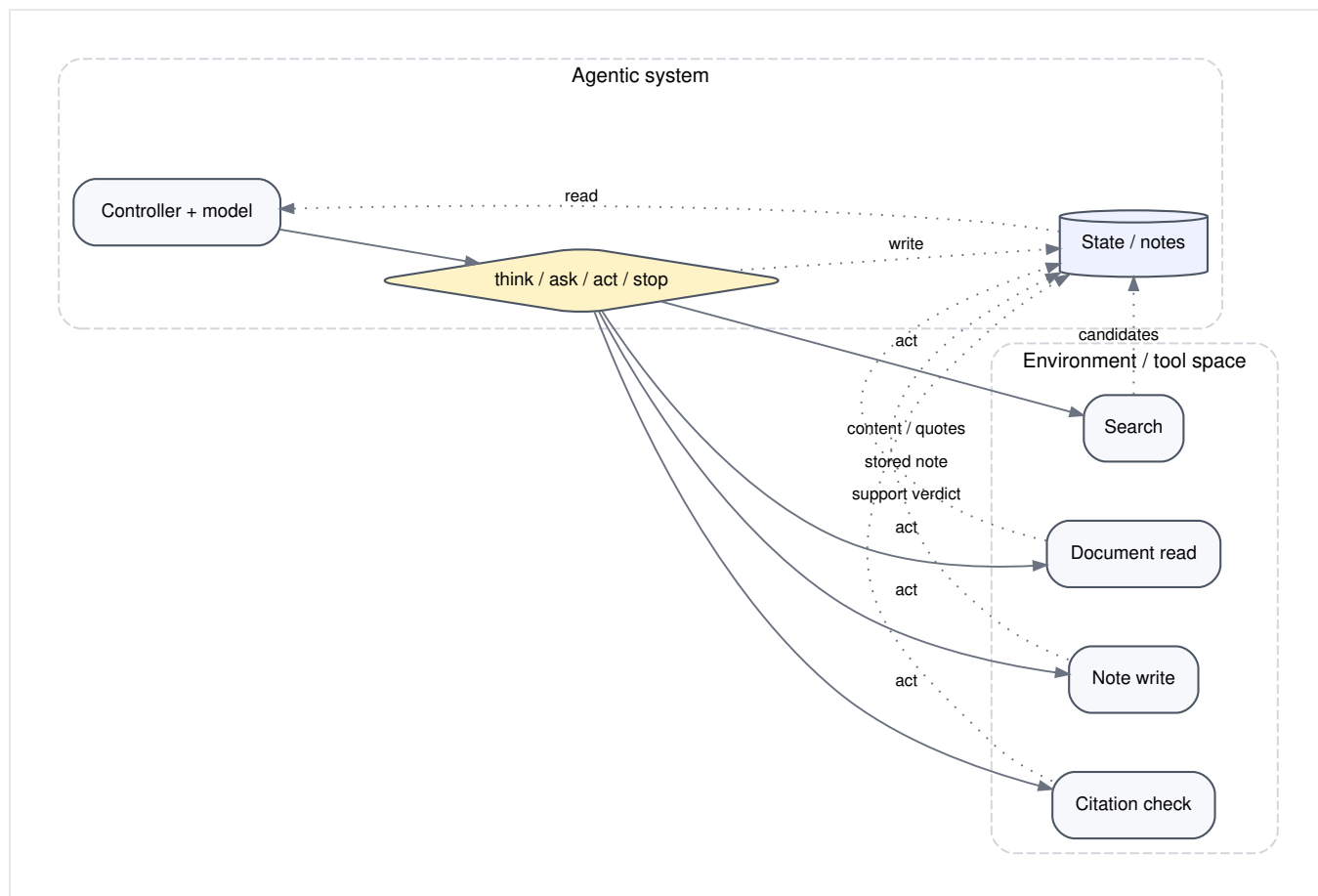


Figure 5.1. Environment coupling for the running case. The environment boundary matters because the agent can know only through its observation channels and can affect only through available actions.

Grounding primer

Grounding is not a synonym for sounding specific. It is the coupling of claims or decisions to observations or evidence.

For the literature-review assistant, the following are different:

- “I know this paper supports answer-level citation display because I read the relevant section.” — potentially grounded.
- “This probably supports answer-level citation display because papers in this area usually do.” — ungrounded.
- “The search tool returned this paper.” — grounded observation about retrieval, not about the paper's content.
- “The citation-check tool found a source link.” — grounded observation about a mapping, not automatic proof that the claim is supported.

This distinction is why tool outputs must be interpreted, not merely repeated (Nakano et al., 2021; Menick et al., 2022).

Asking can be better than acting

The `ask` action is often underused because it feels less autonomous. In bounded systems, that is backwards. Asking is sometimes the most intelligent control move available.

In the running case, imagine the user's phrase “for students” remains ambiguous after initial interpretation. If that ambiguity changes which papers count as relevant, then a clarifying question is better than issuing a broad search. A broad search would create noisy candidates, weak notes, and wasted synthesis effort. One small clarification can save many later corrective actions.

Side effects matter

Tool use is not epistemically neutral. Even in an information-centric system, tools can have side effects:

- writing notes changes future retrieval and control,
- logging failed queries can anchor future search behavior,
- editing a draft changes what later citation checks operate over,
- a user clarification becomes part of the operative task specification.

A strong architecture therefore treats tool outputs as observations and tool side effects as state changes.

Worked example: the right next move

Suppose the literature-review assistant has already issued one search query and received mostly generic tutoring-agent papers. It now faces three candidate next moves.

1. **Think**: refine the query in light of the mismatch.
2. **Ask**: clarify whether tutoring assistants count.
3. **Act**: issue another search query immediately.

Which is best?

- If the ambiguity about tutoring versus literature assistants is unresolved and would strongly change relevance, **ask** is best.
- If the user intent is already clear but the first query was simply too broad, **think** followed by **act** is best.
- If enough evidence already exists and only claim support is missing, **act** with citation check may be best.

The important point is that “use a tool” is not the default. It is one choice among several.

Pseudocode example

```
procedure CHOOSE_ACTION(G, S_t, O_t):
  if consequential_scope_ambiguity(G, S_t):
    return ask
  if success_criteria_met(G, S_t):
    return stop
  if evidence_gap_detected(S_t):
    return act_with(citation_check)
  if observation_needed_for_progress(S_t):
    return act_with(search_or_read)
  return think
```

This pseudocode is deliberately policy-like rather than model-centric. The architecture can consult the model while making the choice, but the *choice itself* is the design object.

Cumulative artifact update

The literature-review assistant now gains a tool contract sheet and an explicit action-choice vocabulary.

Artifact	Current content
Tool contracts	search, document read, note write, citation check
Action set	think, ask, act, stop
Grounding primer	claims must be tied to observations, not merely plausible text

These artifacts create the concrete action space on which planning will operate in the next chapter.

What breaks if this component is missing or misused?

If the architecture has no explicit action-selection logic, one of two degenerate patterns appears:

- **tool reflex:** uncertainty triggers tool use by default, even when clarification or stopping would be better;
- **reasoning reflex:** uncertainty triggers more internal generation even when evidence must come from the environment.

If tool contracts are underspecified, additional failures appear:

1. the system misreads what a tool output actually guarantees,
2. side effects are ignored,
3. the environment boundary becomes conceptually invisible,
4. ungrounded claims inherit false authority because they were adjacent to a tool result.

Failure modes, limitations, and misconceptions

- **Tool use as default.** Tools are options, not automatic solutions.
- **“A successful tool call means the task succeeded.”** No. Tool success is only one ingredient in task success.
- **Ignoring side effects.** Writing, logging, or editing changes future control.
- **Treating all textual tool output as grounded evidence.** A tool can return text that still requires interpretation and verification.
- **Forgetting the environment boundary.** The system cannot know or affect what its tools do not expose (Schick et al., 2023; Nakano et al., 2021).

Chapter summary

This chapter connected the architecture to the environment. The key lesson is that action selection is not about “having tools” but about choosing among thinking, asking, acting, and stopping under explicit contracts. Tool use changes both capability and epistemic

position. Grounding begins here: claims gain status only when appropriately linked to observations.

The next chapter asks how the system should structure work across multiple action choices. Once the action space exists, planning becomes possible.

References and further reading

- (Schick et al., 2023) for models learning when to use tools.
- (Nakano et al., 2021) for browser-mediated evidence gathering.
- (Menick et al., 2022) for supporting answers with explicit evidence.
- (Yao et al., 2022) for reasoning-acting traces that make action choice visible.

Architecture delta

- **Added:** explicit `think / ask / act / stop` control choices and tool contracts.
- **Failure addressed:** vague tool use that confuses action success with task progress.
- **New risk introduced:** overactive tooling or misplaced trust in tool-returned text.

Chapter 6 — Planning, Decomposition, and Replanning

AA-S06 — Planning, decomposition, and search intuition

Design pressure from the running case

The literature-review assistant now knows how to choose actions and use tools. The next pressure is structural.

One version of the system responds in one shot. It searches little, reads little, and tends to flatten the topic into generic summary. Another version decomposes too aggressively into brittle micro-steps: “search query 1, then query 2, then open exactly eight papers, then write section A, then section B.” It breaks the moment the evidence does not match the script.

The architectural question is not whether planning is good in the abstract. It is **when structured future guidance improves control enough to justify its cost**.

Chapter overview

This chapter presents planning as a selective control strategy. It introduces hierarchical decomposition, plan representations, replanning triggers, and a light search intuition built around branching, lookahead, backtracking, and the value of information. The literature-review assistant is used to compare three modes:

- direct response,
- fixed plan,
- iterative replanning.

Core concepts and definitions

A **direct response** architecture chooses and executes the next move without constructing a substantial explicit future structure.

A **plan** is intended future structure. It may be represented as a list, tree, checklist, or dependency-aware graph.

Decomposition breaks a task into subgoals or subproblems.

Replanning revises the current plan because observations show that the previous plan no longer fits the task or world state.

A **planning trigger** is the condition that makes explicit planning worth doing: long horizon, uncertainty, dependencies, scarce resources, or the need to compare branches.

Nearest confusable distinction — plan vs execution trace

A plan says what the system intends to do next. A trace records what it actually did. A checklist does not become a trace just because it was written down. This distinction matters even more once replanning begins, because the current plan may change while the trace stays fixed.

Intuition first

Planning is a control choice, not a synonym for intelligence.

A task deserves explicit planning when future steps depend on one another enough that thinking only one move ahead is costly. The literature-review assistant shows several reasons this can happen:

- the review has multiple deliverable components,
- reading choices affect later synthesis quality,
- early evidence may reveal that the original scope was too broad,
- and verification steps should not be forgotten at the end.

But planning also costs something. It consumes time, can overcommit the system to an early picture of the task, and may create a false sense of progress if the plan is mistaken for the work itself. So the right question is not “should agents plan?” It is “what form of planning, if any, fits this task shape?”

Technical breakdown

Direct response, fixed plan, and replanning

The running case makes the contrast concrete.

Direct response. The system performs a little retrieval and writes immediately. This can work for short, familiar tasks. It fails when evidence must be gathered and compared in a structured way.

Fixed decomposition. The system plans once at the beginning, perhaps:

1. issue search queries,
2. read eight papers,
3. fill the comparison matrix,
4. write the review,
5. citation-check,
6. stop.

This is better when the task is moderately structured and the evidence is unlikely to surprise the system. But if the first queries reveal that answer-level citation is much rarer than expected, the fixed plan may still force an unhelpful “read eight papers” quota.

Iterative replanning. The system begins with a plan but revises it after important observations. This is often the best fit for bounded literature review because evidence quality and category boundaries become clearer as the system searches.

Plan representations

A plan can be represented in several lightweight forms.

Representation	Example in the running case	Strength	Weakness
List		simple, readable	weak about dependencies

Representation	Example in the running case	Strength	Weakness
	search → screen → read → synthesize → verify		
Checklist	“query refined?”, “matrix schema updated?”, “citation check run?”	good for completion control	poor for branching
Tree	search branch A vs branch B, then read branches	makes alternatives visible	can overcomplicate small tasks
Dependency-aware structure	do not synthesize final claims until enough verified notes exist	captures preconditions	harder to author and maintain

For beginners, the important lesson is not which formalism is best. It is that different plan shapes fit different tasks.

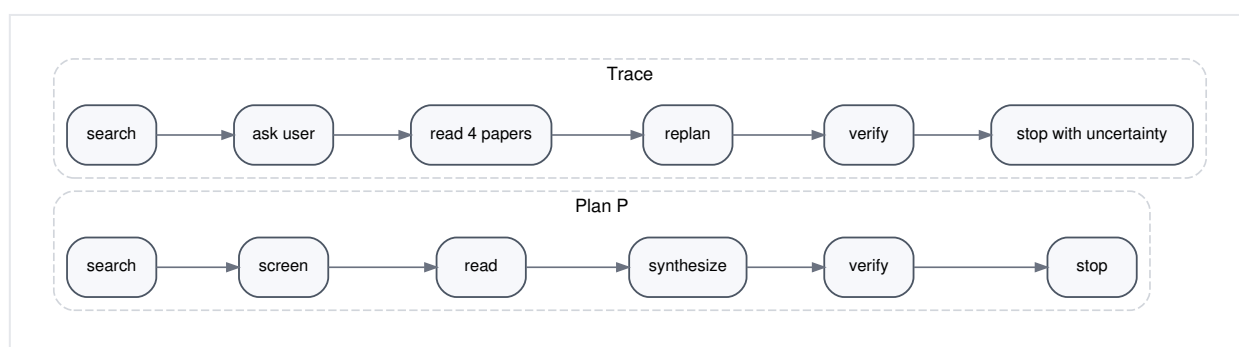


Figure 6.1. Plan versus trace. The plan expresses intended future structure; the trace records realized past behavior. Replanning changes the former, not the latter.

Search intuition without formal algorithms

Planning often benefits from a little search intuition, even in non-symbolic systems (Russell and Norvig, 2021). Four ideas matter most here.

Branching. There may be multiple plausible next paths. For the running case, one query family may focus on “citation-grounded QA,” another on “verified quotes,” and another on “evidence-backed generation.”

Lookahead. Before committing to a branch, the system can estimate which branch is more likely to produce useful evidence.

Backtracking. If a branch fails, the system can return to an earlier choice rather than forcing the failed path.

Information value. Sometimes the best next action is one that reduces uncertainty rather than one that immediately produces a final artifact.

These are not special to symbolic AI. They are general control intuitions.

A toy search fork

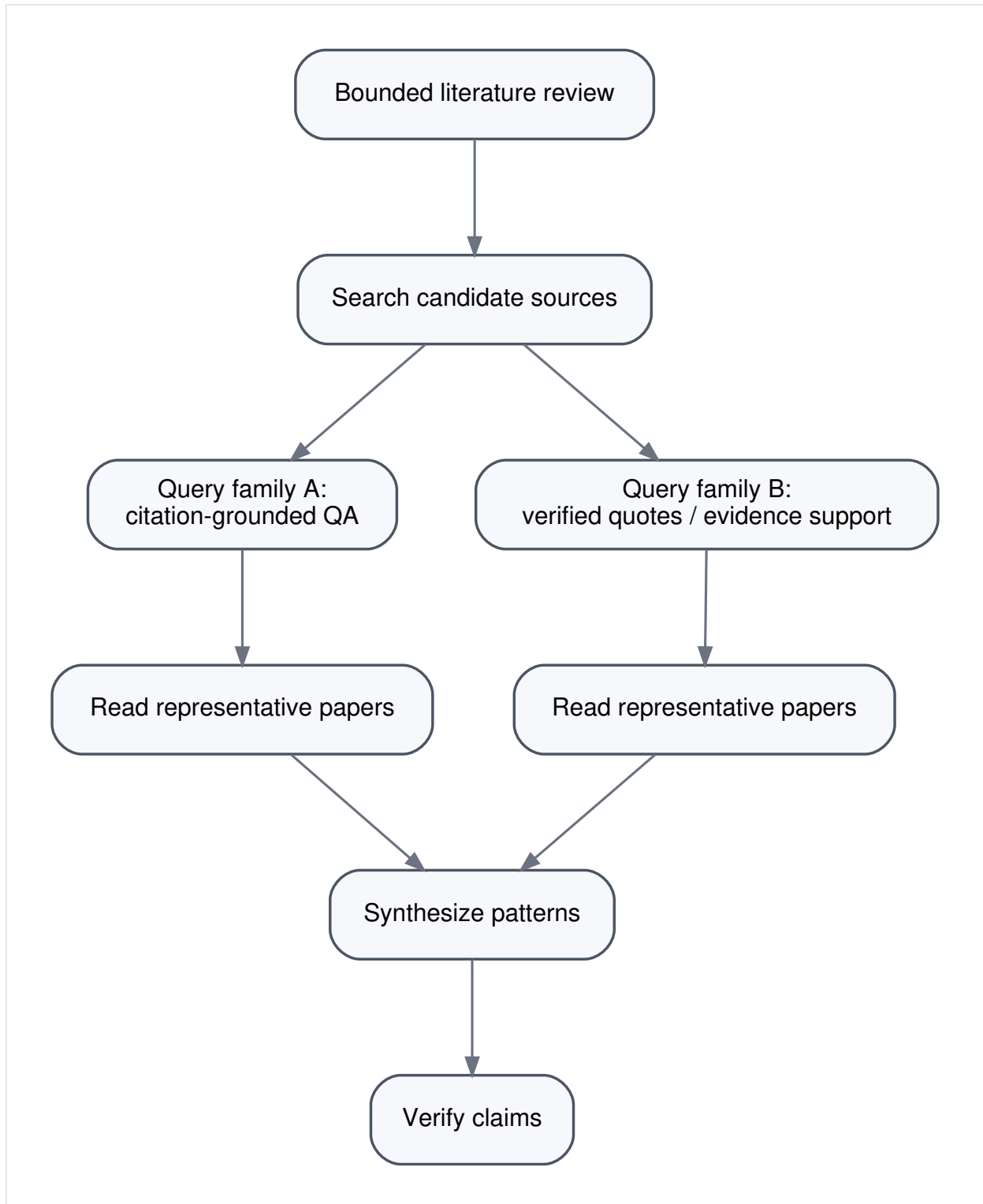


Figure 6.2. A small toy search fork. The point is not algorithmic optimality but the intuition that alternative query branches can be compared, abandoned, or revisited.

Imagine the assistant considers two initial query branches:

- **Branch A:** “citation-grounded question answering assistants students”
- **Branch B:** “verified quotes educational QA systems”

A quick lookahead reveals that Branch A returns many generic tutoring agents with retrieved context but little explicit answer citation. Branch B returns fewer results but more direct evidence about answer support. A planning-aware system may start with Branch B while keeping A in reserve for coverage.

That is planning in the broad sense used here: representing alternatives and making a controlled choice.

Replanning triggers

Replanning is justified when observations change the assumptions behind the current plan. Common triggers in the running case are:

- initial search results are too broad or too sparse,
- a new architecture distinction emerges that changes the comparison matrix,
- important sources turn out to be inaccessible,
- evidence contradicts an earlier working hypothesis,
- or the stop threshold is clearly unreachable within the current bounds.

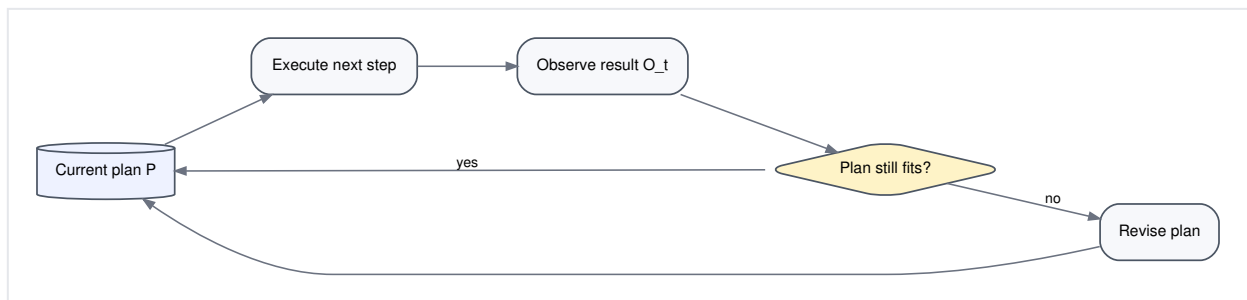


Figure 6.3. Replanning loop. The plan is revised only when observations show that the current plan no longer fits the task or the evidence.

The presence of a replan loop does not mean the system should replan all the time. Frequent needless replanning is its own failure mode.

Over-fragmentation

One of the most common beginner mistakes is to decompose too much. The plan becomes a brittle micro-script:

1. search exactly three times,
2. open exactly eight abstracts,
3. write exactly one note per paper,
4. fill exactly four matrix columns,
5. draft exactly five paragraphs.

This looks organized but often destroys adaptability. Good decomposition preserves decision-relevant structure while leaving room for runtime judgment.

Worked example: three versions of the same task

Consider the same bounded literature-review request.

One-shot response. The system writes an answer after minimal evidence gathering. It is fast but shallow. It often misses category boundaries and uncertainty.

Fixed decomposition. The system creates an initial plan and follows it closely. It is good at ensuring that all required deliverables appear. It struggles when new evidence changes what “relevant” means.

Iterative replanning. The system begins with a skeleton plan:

1. search broadly,
2. identify candidate architecture families,
3. read representative papers,
4. refine matrix dimensions,
5. verify claims,
6. stop or hand off.

After two reads, it notices that “evidence presentation” is more important than it expected and that one intended family has too little evidence. It revises the plan: fewer papers from the weak family, more targeted search around verified quotes, and an added uncertainty note in the final deliverable.

That is exactly the kind of bounded adaptation planning should support.

Pseudocode example

```

procedure PLAN_ACT_REPLAN(G, S_t):
  if no_current_plan(S_t):
    P <- make_initial_plan(G, S_t)

  while true:
    next_step <- choose_from_plan(P, S_t)
    O_t <- execute_or_observe(next_step)
    S_t <- update_state(S_t, O_t)

    if stop_condition_met(G, S_t):
      return finalize(S_t)

    if replan_triggered(P, S_t, O_t):
      P <- revise_plan(G, P, S_t)

```

The architecture question is not how fancy `make_initial_plan` is. It is when `revise_plan` is warranted.

Cumulative artifact update

The literature-review assistant now gains an explicit plan artifact `P`.

Artifact	Current content
Current plan <code>P</code>	a bounded decomposition from search to verification and stop
Replanning triggers	

Artifact	Current content
	broad or sparse evidence, changed categories, inaccessible sources, contradiction, unreachable stop threshold

This artifact will later interact with feedback and stopping rules.

What breaks if this component is missing or misused?

If planning is missing on a task that needs it, the architecture becomes short-sighted. It may repeatedly choose locally plausible actions without preserving overall structure.

If planning is misused, different failures appear:

1. **plan worship:** the system treats the plan as truth rather than revising it,
2. **over-fragmentation:** brittle micro-steps replace judgment,
3. **trace confusion:** written intent is mistaken for completed work,
4. **replanning churn:** the plan changes so often that work never stabilizes.

The running case needs planning, but only in a bounded, revisable form.

Failure modes, limitations, and misconceptions

- **“Planning is always better.”** No. Planning adds overhead and can overfit early assumptions.
- **Confusing plan with execution trace.** A drafted structure is not evidence of progress.
- **Over-fragmenting tasks.** Too many micro-steps can turn adaptation into fragility.
- **Treating search intuition as belonging only to symbolic AI.** Branching, lookahead, and backtracking are general control ideas, not property of one historical tradition (Russell and Norvig, 2021; Huang et al., 2022).
- **Ignoring information-gathering value.** Some actions are worthwhile because they sharpen later choices, not because they immediately produce deliverables.

Chapter summary

Planning is a selective control strategy over an action space. The right comparison is not “planned” versus “intelligent,” but direct response versus fixed decomposition versus iterative replanning. The literature-review assistant benefits from a small, revisable plan because evidence quality and category boundaries emerge during the run.

The next chapter asks how the architecture decides whether its current plan, notes, or intermediate claims are trustworthy enough to keep following. That is where grounding, feedback, and structural error analysis enter.

References and further reading

- (Wang et al., 2023) for explicit planning prompts in reasoning tasks.
- (Yao et al., 2023) for branching and deliberate exploration over intermediate thoughts.

- (Huang et al., 2022; Huang et al., 2022) for planning and feedback in embodied settings.
- (Russell and Norvig, 2021) for classical search and planning intuition as neighboring background.

Architecture delta

- **Added:** an explicit plan P , plus replanning triggers.
- **Failure addressed:** purely local next-step choice on tasks with dependencies and evidence uncertainty.
- **New risk introduced:** brittle or overactive planning that mistakes structure for progress.

Chapter 7 — Grounding, Feedback, Reflection, and Structural Error Analysis

AA-S07 — Grounding, feedback, reflection, and structural error analysis

Design pressure from the running case

The literature-review assistant now has goals, state, memory, tools, and a revisable plan. It can still fail in a familiar way: it keeps moving, and the movement looks organized, but the output is not trustworthy.

In one run, the system writes a polished synthesis claiming that “recent systems usually support answers with verified quotes.” A citation-check step confirms that the cited papers exist, yet closer inspection shows that only some papers actually provide quote-level support. The architecture had a generic reflection step—“review your answer for quality”—but no evidence-bearing verification tied to the claim.

In another run, a stale concept note from a prior review leaks into the current one, and the system confidently overstates a pattern that the present evidence does not support.

These failures are not solved by telling the model to be more careful. They require better coupling between claims, evidence, feedback, and control.

Chapter overview

This chapter deepens grounding and introduces feedback as the broader class of signals that can change what the system does next. It distinguishes reflection from verification and uses a structural error taxonomy to diagnose failures at the level of goals, state, memory, planning, action, grounding, and evaluation.

The central lesson is simple: **reflection helps only when it is answerable to evidence and linked to corrective action.**

Core concepts and definitions

A **grounding gap** exists when a claim or decision is insufficiently tied to observations or evidence.

An **evidence gap** exists when the architecture lacks the observations needed to justify the current claim or next step.

Feedback is any signal that can alter control: tool results, environment responses, user corrections, internal checks, or explicit evaluation heuristics.

Reflection is internal review of the current output, plan, or state.

Verification is evidence-bearing checking. It tests whether a claim, note, or decision is supported by appropriate observations.

A system may retain **residual uncertainty** after action. Good architectures surface that uncertainty rather than converting it into unwarranted confidence.

Nearest confusable distinction — reflection vs verification

Reflection asks, “Does this look right, complete, or coherent?” **Verification** asks, “What evidence supports this claim or decision, and does the evidence actually match it?” Reflection can improve style or structure. Verification is what binds outputs to the world.

Intuition first

A generic self-critique prompt can sometimes improve an output (Madaan et al., 2023). But in architecture, the important question is not whether self-critique occasionally helps. It is whether the system has access to **the right evidence** and whether the critique can cause **the right repair**.

The literature-review assistant makes this concrete. Suppose its draft says, “Most recent educational QA assistants provide explicit citations in final answers.” A generic reflection step may ask whether the sentence is too broad. The system might even soften it a little. But unless the architecture can inspect the note set, map each clause to supporting sources, and detect missing or contradictory evidence, the reflection is still floating above the actual task.

Grounding and feedback make reflection answerable.

Technical breakdown

Feedback sources

For bounded literature review, useful feedback can come from four sources.

1. **Tools.** Search quality, document accessibility, citation-check results.
2. **Environment.** Missing documents, changed source availability, contradictory content.
3. **User.** Clarifications, corrections, changed priorities.
4. **Internal checks.** Coverage tests, support mapping, freshness checks, contradiction detection.

These signals are only useful if the architecture routes them to decisions. A system that logs feedback but never lets it alter control is not truly feedback-driven.

Grounding claims to observations

A helpful architectural discipline is to treat each substantive claim as needing an evidence path.

For the running case, a claim such as “verified-quote systems trade coverage for stronger citation support” should ideally connect to:

- specific read papers,
- note fields capturing the relevant architectural property,
- and perhaps a contradiction or limitation field if the evidence is mixed.

This does not require formal proof machinery. It requires the architecture to preserve a usable mapping between claims and sources.

Structural error taxonomy

To diagnose failures without drifting into benchmark-heavy evaluation, this book uses a structural taxonomy.

Error class	Typical symptom in the running case	Likely root cause	Typical redesign
Goal error	review answers the wrong question	task specification too vague or misunderstood	clarify or recompile <code>G</code>
State error	exclusion rule disappears mid-run	criterion lived only in context	write rule into <code>S_t</code>
Memory error	stale note shapes current synthesis	poor freshness or contamination policy	separate stores, add freshness checks
Planning error	system follows an obsolete plan	weak replanning trigger	revise replan conditions
Action error	too many searches, too little reading	poor <code>think/ask/act/stop</code> choice policy	improve action selection
Grounding error	claim sounds plausible but lacks support	no claim-evidence mapping	add verification step
Evaluation error	system declares success too early	weak success or stop criteria	tighten success checks

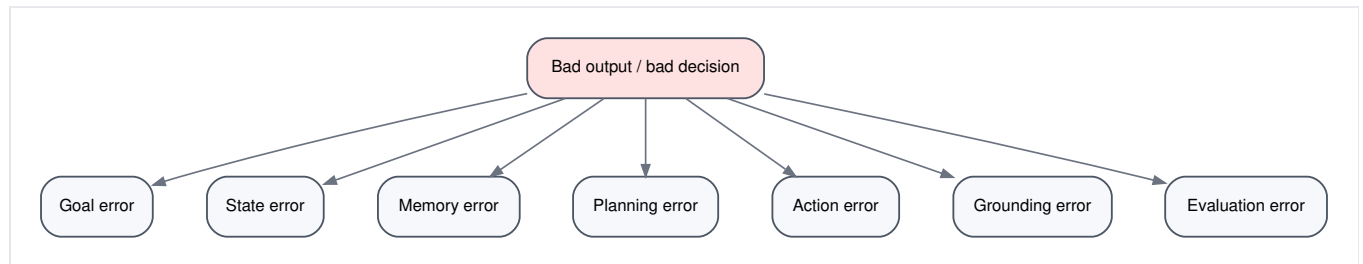


Figure 7.1. Structural error taxonomy for bounded agent analysis. The same bad output can arise from different root causes; diagnosis should start with architecture, not blame assignment.

This taxonomy matters because a weak final output does not tell you which component failed. “The model hallucinated” is often too coarse to be useful. The architecture may have omitted the evidence path entirely.

Failure trace 1: weak evidence checks

Consider this trace fragment.

t	observation	action	result
5	four notes mention retrieval; one note mentions verified quotes	think	draft synthesis sentence: “recent systems usually support answers with verified quotes”
6	generic reflection prompt returns “looks plausible”	think	sentence kept
7	citation-check finds cited papers exist in note set	act	sentence treated as supported

The architecture failed because its verification target was too weak. It checked whether cited sources were present, not whether the **claim** matched the sources.

A better design would require the synthesis sentence to list supporting note identifiers and would reject or soften the claim if the mapping was insufficient.

Failure trace 2: stale note contamination

Now consider a different run. The system retrieves a semantic note from an older review on medical QA systems that says “retrieval augmentation is usually presented as grounding.” The current task is narrower: answer-level citation support for students. The old note biases the synthesis, and the system overgeneralizes.

This is not primarily a generation problem. It is a memory-policy problem. The wrong abstraction was retrieved for the wrong scope.

Designing informative feedback loops

A feedback loop is useful only if it tells the system something actionable. Generic self-critique often fails that test. More informative loops in the running case include:

- **coverage check:** have enough representative architecture patterns been read?
- **support check:** does each substantive draft claim map to at least one verified source?
- **freshness check:** is this retrieved note or template appropriate for the current scope?
- **contradiction check:** do source notes disagree on a pattern or limitation?
- **completion check:** do current artifacts satisfy `G_review`?



Figure 7.2. Revised feedback loop. Reflection is retained, but it is driven by explicit support checks, contradiction checks, and completion checks rather than generic self-critique alone.

Notice what changes here. Feedback is not merely commentary on the draft. It becomes a trigger for replanning, additional reading, claim revision, clarification, stopping, or handoff.

Reflection after evidence, not instead of evidence

Reflection still has a role. It can help the assistant notice missing dimensions in the comparison matrix, weak transitions, overclaiming language, or unresolved ambiguity. But its value is highest **after** the architecture has exposed the relevant evidence and uncertainty.

In that sense, reflection is often best treated as a consumer of structured evidence, not a substitute for it (Shinn et al., 2023; Madaan et al., 2023).

Worked example: redesigning the loop

Suppose the literature-review assistant finishes a draft and runs two checks.

1. **Support mapping check.** Two sentences in the summary have no strong note-level support.
2. **Contradiction check.** Three papers disagree on whether inline citations improved factual trust for student use cases.

Now the architecture has informative options:

- revise unsupported sentences,
- read one additional paper targeted at the contradiction,
- surface the disagreement explicitly,
- or hand off if the conflict exceeds the system's competence or available evidence.

The key difference is that the feedback is tied to specific corrective actions. Generic self-critique alone would not tell the system which move is justified.

Pseudocode example

```
procedure VERIFY_AND_REFLECT(draft, S_t):
  unsupported <- find_claims_without_support(draft, S_t.notes)
  contradictions <- detect_contradictions(S_t.notes)
  stale_items <- find_stale_retrievals(S_t.memory_reads)

  if unsupported or contradictions or stale_items:
    return feedback_packet(unsupported, contradictions, stale_items)

  reflections <- reflect_on_structure_and_clarity(draft, supported_by=S_t.notes)
  return feedback_packet(reflections)
```

The important output is the **feedback packet**, not the introspective prose. The packet must drive control.

Cumulative artifact update

The literature-review assistant now gains explicit feedback rules and a structural failure matrix.

Artifact	Current content
Feedback rules	support check, contradiction check, freshness check, completion check
Failure matrix	structural taxonomy across goals, state, memory, planning, action, grounding, evaluation

This chapter also revisits error analysis as light evaluation: the point is to locate root causes and redesign the loop, not to produce benchmark tables.

What breaks if this component is missing or misused?

If grounding and feedback are weak, the architecture becomes output-oriented rather than evidence-oriented. It keeps producing text without a strong reason to trust or revise it.

If reflection is misused, common failures are:

1. **generic critique with no evidence path**,
2. **confidence inflation**, where polished prose is mistaken for verified support,
3. **misdiagnosis**, where every failure is blamed on the model instead of the architecture,
4. **unchecked progress**, where successful intermediate steps are mistaken for task success.

Failure modes, limitations, and misconceptions

- **“Just add reflection.”** Reflection without evidence often produces nicer wording, not more trustworthy outputs.
- **Confidence is not correctness.** A strong model can produce confident, coherent, unsupported synthesis.
- **Blaming the model for architectural failures.** Missing state, stale memory, or weak verification are system-design problems.
- **Treating any successful intermediate step as final success.** A clean note, successful search, or technically valid citation link does not complete the task (Menick et al., 2022; Nakano et al., 2021).

Chapter summary

This chapter taught the system how to get corrected rather than merely how to keep moving. Grounding ties claims and decisions to observations. Feedback is the broader signal class that changes control. Reflection is useful, but only when it consumes evidence and points to corrective action. Structural error analysis helps diagnose failures at the right level: goal, state, memory, planning, action, grounding, or evaluation.

The next chapter closes the control story by asking how the system decides whether to continue, ask, stop, or hand off.

References and further reading

- (Menick et al., 2022) for explicit evidence support.
- (Nakano et al., 2021) for evidence-gathering with references.
- (Madaan et al., 2023; Shinn et al., 2023) for iterative reflection methods and their strengths and limits.
- (Huang et al., 2022) for feedback shaping control in interactive settings.

Architecture delta

- **Added:** evidence-bearing verification, feedback rules, and a structural error taxonomy.

- **Failure addressed:** generic self-critique that cannot diagnose or repair unsupported outputs.
- **New risk introduced:** overchecking or poorly designed feedback that slows the run without adding actionable information.

Chapter 8 — Stopping, Clarification, Handoff, and Bounded Autonomy

AA-S08 — Stopping, handoff, and bounded autonomy

Design pressure from the running case

A common naive version of the literature-review assistant has no principled ending. It keeps searching because more evidence might exist, or keeps rewriting because the draft might improve. Another version stops too early after finding a handful of papers, even though the task specification asked for a comparison matrix and explicit uncertainty notes. A third version never asks the user anything and treats every ambiguity as a problem to solve internally.

These are not production quirks. They are conceptual design failures. A strong architecture defines not only how to continue, but when to ask, when to stop, and when to hand off.

Chapter overview

This chapter completes the core control story. It distinguishes completion from abandonment, introduces sufficiency and uncertainty thresholds, and frames clarification and handoff as first-class architectural moves. It also connects these ideas to **bounded autonomy**: permission to continue acting under explicit limits.

By the end of the chapter, you should be able to read an agent description and ask: how does it know when enough is enough, when ambiguity requires the user, and when continuing would exceed its designed authority?

Core concepts and definitions

Completion means the task specification's success criteria have been met.

Abandonment means the system has stopped without satisfying the task, often because it cannot make justified progress.

A **sufficiency threshold** is the point at which available evidence and artifacts are enough for completion.

An **uncertainty threshold** is the point at which uncertainty is too high to continue without clarification, further evidence, or handoff.

Clarification is controlled user re-entry to resolve consequential ambiguity.

Handoff is transfer of control or responsibility because the current system should not continue alone.

Bounded autonomy means the system may continue acting under explicit limits. It does not mean open-ended initiative.

A helpful conceptual distinction is between **reversible** and **irreversible** actions. Information gathering and draft writing are usually reversible. Publishing, deleting,

committing, or notifying may be irreversible. Even in an information-centric assistant, the action class shapes autonomy policy.

Nearest confusable distinction — completion vs stopping vs handoff

Completion is a property of the task state: the success criteria are met. **Stopping** is a control decision: the system will not continue. **Handoff** is a special kind of stopping in which work is escalated or returned to a human or another component. A system can stop because it completed the task, or it can stop because it should not continue.

Intuition first

Beginners often treat stopping as an afterthought: once the “interesting” parts of search, memory, and planning are built, surely the architecture can just finish somehow. In practice, stopping is what converts uncontrolled activity into bounded behavior.

The literature-review assistant needs stopping logic for at least four recurring situations:

1. **sufficient completion:** enough evidence has been gathered and the deliverable meets the success criteria;
2. **thin evidence:** the assistant cannot responsibly continue because the source base is too weak;
3. **ambiguous scope:** the assistant can continue only after clarification;
4. **conflicting sources:** the assistant can describe the disagreement, but beyond some point it should hand off rather than invent resolution.

Stopping is therefore not only about endings. It is about protecting the architecture from unjustified continuation.

Technical breakdown

Continue vs ask-user vs stop vs handoff

At a high level, the control decision now becomes:

- **continue** if the next move is justified and bounded;
- **ask-user** if consequential ambiguity blocks justified continuation;
- **stop** if the task is complete or further action is not warranted;
- **handoff** if the system should not continue on its own.

This choice is task-relative. A short exploratory search task may tolerate thin evidence more easily. A literature-review assistant that promises evidence-backed synthesis should be stricter.

Sufficiency thresholds

A sufficiency threshold is not “perfect coverage.” It is a task-relative point of adequacy.

For the running case, a reasonable sufficiency threshold might require:

- a representative set of 8–12 papers if available, or fewer with explicit uncertainty if the field is small;

- coverage of at least three architecture patterns if the literature supports that many;
- a completed comparison matrix for all included papers;
- support mapping for the main synthesis claims;
- explicit note of unresolved disagreements or thin spots.

The threshold should be high enough to protect task quality and low enough to allow bounded completion.

Uncertainty thresholds

The system should not continue indefinitely under unresolved uncertainty. Typical uncertainty thresholds in the running case include:

- the scope remains ambiguous after available internal reasoning,
- accessible evidence is too thin to support the promised deliverable,
- retrieved notes disagree in ways that the assistant cannot adjudicate with available tools,
- or the remaining uncertainty would materially change the output's framing.

When these thresholds are crossed, clarification or handoff becomes preferable to continued generation.

Clarification as user re-entry

Clarification is best when the ambiguity is:

- consequential,
- answerable by the user,
- and likely to reduce wasted work.

For the literature-review assistant, a good clarification case is:

Should the review include tutoring assistants that retrieve evidence for internal use, or only systems that present citations or quotes directly in final answers?

That question changes what counts as relevant. It is therefore better to ask than to proceed under a hidden assumption.

Handoff conditions

Handoff is appropriate when:

- the system lacks the authority to continue,
- the evidence is too weak for the promised claim strength,
- conflict among sources requires domain interpretation beyond the architecture's scope,
- or the action would move from reversible information work into a more consequential domain.

For the running case, a clear handoff case is this: the assistant finds conflicting claims about the educational trust effects of explicit citations, and the user expects policy or

pedagogical recommendations. The assistant can summarize the conflict, but a human expert should interpret the implications.

Stop-policy table for the running case

Scenario	Recommended decision	Why
Enough representative papers read, matrix complete, main claims supported	stop with final review	sufficiency threshold met
Evidence is thin after targeted search and scope is clear	stop with bounded review plus explicit uncertainty, or hand off if promised deliverable cannot be met	continuing may only inflate confidence
“For students” remains ambiguous and changes inclusion criteria	ask user	ambiguity blocks justified continuation
Sources conflict on an interpretive recommendation outside system authority	hand off	escalation is safer than forced resolution
More search may add small marginal value but success criteria are already met	stop	bounded autonomy requires restraint

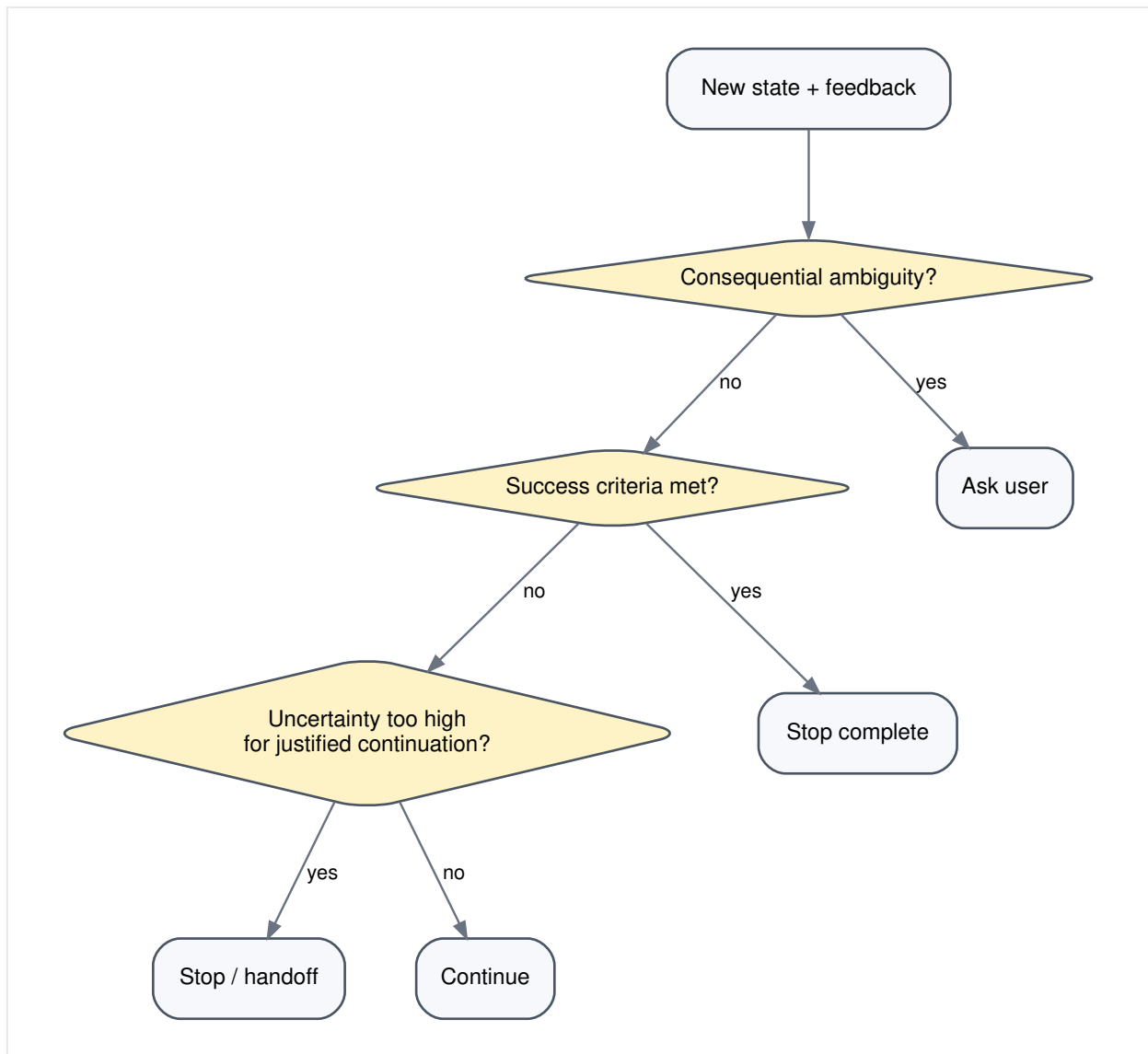


Figure 8.1. A simple stop-decision tree. Good architectures do not merely know how to continue; they also know when continuation is no longer justified.

Bounded autonomy

Bounded autonomy is a design decision about what the system may do without user confirmation.

For the running case, a reasonable autonomy boundary is:

- allowed without confirmation: search, read, take notes, refine the plan, draft the review, run verification checks;
- requires clarification: materially ambiguous scope decisions;
- requires handoff or approval: claims that exceed available evidence, recommendations beyond synthesis, or irreversible downstream actions.

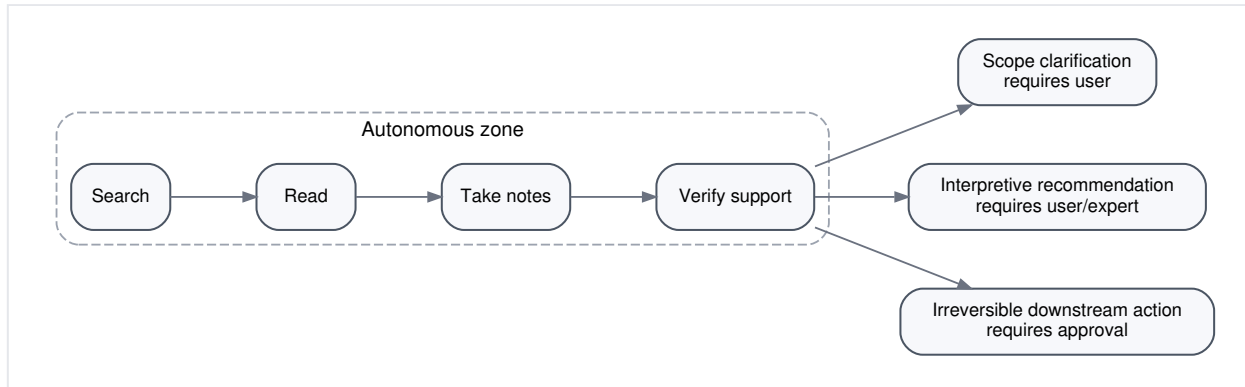


Figure 8.2. Autonomy boundary for the running literature-review assistant. The point of the boundary is not to make the system passive, but to let it act confidently within explicit limits.

Light HCI research on human-AI systems repeatedly emphasizes expectation management, recoverability, and user control (Amershi et al., 2019). This chapter translates that lesson into architecture: user re-entry is part of control, not a fallback embarrassment.

Worked example: writing a stop/handoff policy

Suppose the literature-review assistant has read six strong papers and two uncertain ones. It has enough material to identify three architecture patterns and fill most of the comparison matrix. However, one requested angle—effects on student trust—is supported only weakly and by conflicting studies.

A principled policy could say:

- stop the main review if architecture and grounding claims are sufficiently supported;
- explicitly mark the student-trust section as mixed or thin evidence;
- hand off if the user wants practical recommendations for classroom deployment;
- ask for clarification if the user would rather widen the scope to include broader educational assistants.

That policy respects bounded autonomy. The assistant does not pretend every unresolved issue can be solved by more internal work.

Pseudocode example

```

procedure DECIDE_CONTROL_AFTER_FEEDBACK(G, S_t, feedback):
  if consequential_ambiguity(S_t):
    return ask_user
  if success_criteria_met(G, S_t):
    return stop_complete
  if evidence_too_thin_for_promised_output(G, S_t):
    if user_can_reframe_task(G):
      return ask_user
    else:
      return handoff_or_stop_with_limits
  if conflict_exceeds_authority(S_t):
    return handoff
  return continue
  
```

This is the first place where “authority” enters the book in an explicit way. Not because the assistant is a legal or organizational actor, but because bounded design always includes a notion of what the system is permitted to decide on its own.

Cumulative artifact update

The literature-review assistant now gains a stop and handoff policy.

Artifact	Current content
Stop policy	sufficiency threshold, uncertainty threshold, and scenario-based decisions
Autonomy boundary	what the assistant may continue doing alone and what requires user or expert re-entry

This artifact closes the core loop. The next chapter uses all the accumulated artifacts to compare whole architectures by task shape.

What breaks if this component is missing or misused?

If stopping logic is missing, the system either loops indefinitely or stops arbitrarily. If clarification is missing, the system hides assumptions instead of surfacing them. If handoff is missing, the system may act beyond its justified competence.

Misused stopping logic causes opposite failures:

1. **premature stopping**, where easy-to-meet partial criteria masquerade as completion;
2. **never enough**, where the system keeps searching because perfect certainty is impossible;
3. **clarification aversion**, where the system treats every ambiguity as something to solve internally;
4. **false autonomy**, where the system performs actions that should require approval or escalation.

Failure modes, limitations, and misconceptions

- **Infinite conceptual loops.** Without a stop rule, improvement can be imagined forever.
- **Premature stopping.** A coherent paragraph is not the same as task completion.
- **Treating clarification as failure.** Clarification is a legitimate control action.
- **Treating stopping and handoff as operational detail.** They are part of the architecture's conceptual integrity (Parasuraman et al., 2000; Amershi et al., 2019).

Chapter summary

A strong architecture does not merely continue well. It ends well. Completion, stopping, clarification, and handoff are different control outcomes, and each needs explicit criteria. Bounded autonomy is the design discipline that keeps these outcomes coherent. The next chapter steps back and uses everything built so far to compare whole architectures by task shape and scope.

References and further reading

- (Amershi et al., 2019) for user control, expectation setting, and recoverability in human-AI interaction.
- (Parasuraman et al., 2000) for reasoning about levels of automation and when not to automate further.
- (Xi et al., 2023) for broad agent design patterns in which autonomy and handoff are treated differently.

Architecture delta

- **Added:** stop criteria, clarification rules, handoff conditions, and an autonomy boundary.
- **Failure addressed:** endless continuation or arbitrary stopping.
- **New risk introduced:** thresholds that are either too strict to complete work or too loose to protect against unjustified action.

Part III — Architectural Judgment and Synthesis

Turn component knowledge into comparison, task–architecture fit, scope boundaries, and a defended capstone design.

Chapter 9 — Comparing Architectures by Task Shape and Scope

AA-S09 — Architecture comparison and boundaries

Design pressure from the running case

By now the literature-review assistant has accumulated a substantial architecture: task specification, explicit state, memory policy, tool contracts, a revisable plan, feedback checks, and stop or handoff rules. But that does not mean every bounded information task should receive the same architecture.

If the user only needs three recent survey papers and one sentence on each, a heavy plan-memory-act-reflect-stop system may be overbuilt. If the user needs a careful architecture-first literature review with explicit uncertainty handling, a prompt-only answer is probably underbuilt.

The design pressure in this chapter is therefore one of judgment: **how do we compare architectures by task shape rather than by buzzword?**

Chapter overview

This chapter introduces a task-shape lens with six main dimensions:

- horizon,
- uncertainty,
- memory need,
- tool dependence,
- feedback richness,
- autonomy bound.

It uses that lens to compare three recurring architectures for the same running case—prompt-only interactive, scripted retrieval workflow, and bounded plan-memory-act-reflect-stop loop—and then contrasts them with a shorter-horizon task where the fuller agent is unnecessary.

Core concepts and definitions

Task shape is the structural profile of a task: how long it is, how uncertain it is, how much it depends on tools, what memory it needs, how rich its feedback signals are, and how much autonomy it warrants.

An **architecture pattern** is a recurring control arrangement, such as prompt-only interaction or a bounded adaptive loop.

Task-architecture fit is the degree to which a pattern matches the task's structural demands.

A **scope boundary** marks where this book's topic gives way to neighboring topics such as reinforcement learning, information retrieval internals, symbolic planning, software design patterns, frameworks, or operations.

Nearest confusable distinction — architecture pattern vs implementation framework

An architecture pattern is a conceptual organization of goals, loops, state, memory, action, feedback, and stopping. A framework is a software vehicle that may implement many patterns. This chapter compares patterns, not libraries or products.

Intuition first

Architectural complexity is not a virtue. It is a cost that should be paid only when the task shape warrants it.

The running literature-review assistant is useful precisely because it sits in the middle. It is not as simple as a one-turn answer. It is not as open-ended as a continuously acting assistant with broad autonomy. It requires enough structure to expose trade-offs without drifting into production operations or formal planning theory.

A good analyst therefore asks not “which architecture is most advanced?” but “which architecture is adequate, justified, and no more elaborate than necessary?”

Technical breakdown**Six task-shape dimensions**

A practical comparison lens is:

1. **Horizon.** How many dependent steps are needed?
2. **Uncertainty.** How likely is it that observations will change the path?
3. **Memory need.** Must the system preserve structured information beyond the immediate call?
4. **Tool dependence.** Does good performance require fresh interaction with the world?
5. **Feedback richness.** Are there meaningful signals that can correct the run?
6. **Autonomy bound.** How much independent continuation is justified?

These dimensions do not produce a numeric score. They organize judgment.

Canonical bounded architecture patterns

This book's recurring patterns can be summarized as follows.

Pattern	Brief description	Best fit	Common failure
Prompt-only interactive assistant	user drives the loop; model answers from current context	short, well-scoped, low-dependence tasks	early apparent completeness, weak evidence handling
Scripted retrieval workflow	fixed sequence such as search → read → summarize	stable tasks with predictable structure	brittle when evidence surprises the system
Adaptive tool-using loop	explicit state plus runtime action choice	moderate uncertainty and tool dependence	weak without good stop or feedback rules
Bounded plan-memory-act-reflect-stop loop	explicit task spec, state, memory policy, planning, verification, stop/handoff	medium-horizon tasks with evidence requirements and bounded autonomy	overbuilt for short tasks; policy complexity

Comparison on the running case

The bounded literature-review request is medium horizon, moderately uncertain, tool dependent, moderately memory demanding, rich in possible feedback, and explicitly bounded in autonomy. That profile already suggests that the most minimal architecture will be too weak and the most rigid fixed sequence too brittle.

Table 9.1 compares three required versions of the running case.

Architecture	Horizon fit	Uncertainty handling	Memory fit	Tool use	Feedback and stopping	Overall fit for the running case
Prompt-only interactive	weak	weak	weak	little or ad hoc	largely user-mediated	underbuilt
Scripted retrieval workflow	moderate	weak to moderate	moderate if fields are passed	strong for predictable steps	limited unless explicitly added	workable for narrow variants
Bounded plan-memory-act-reflect-stop loop	strong	strong	strong if policies stay minimal	strong and selective	strong	best fit

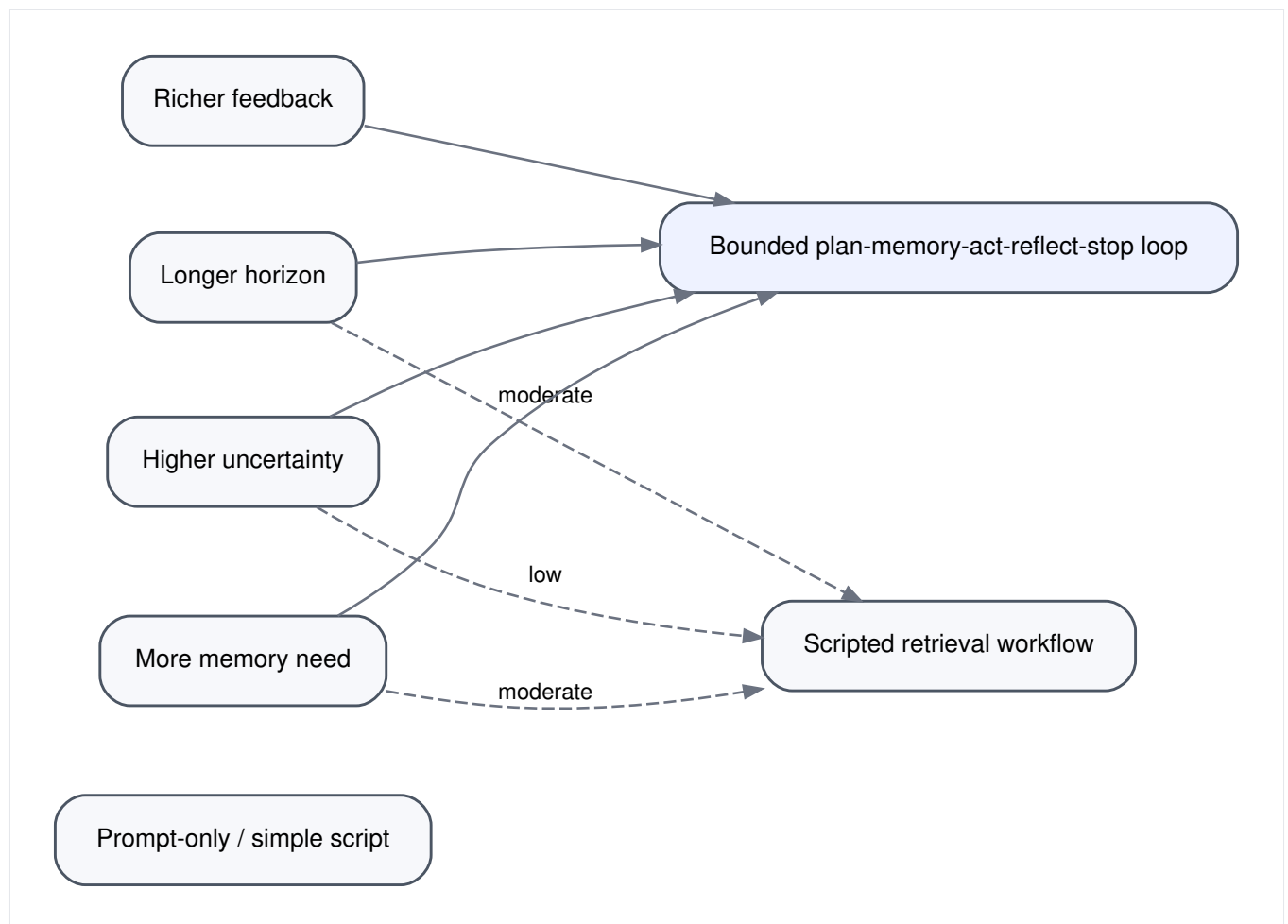


Figure 9.1. Task shape drives architectural fit. The fuller bounded loop sits where horizon, uncertainty, memory need, and feedback richness are all moderate rather than trivial.

The important point is not that the third pattern is “more agentic” and therefore better in every sense. It is that its extra structure directly answers the running task's shape.

A shorter-horizon research task

Now compare a different task:

Find three recent survey papers on retrieval-augmented generation and list one sentence on what each surveys.

This task is shorter horizon, lower uncertainty, lower memory need, and usually lower autonomy risk. A scripted retrieval workflow or even prompt-only plus one retrieval step may be the better fit. Adding elaborate persistent memory, replanning, and handoff logic may be unnecessary overhead.

This example matters because it prevents a common beginner mistake: once a rich architecture is learned, it is tempting to use it everywhere.

Why simpler architectures are sometimes better

Simpler architectures can be better because they are:

- easier to reason about,
- less prone to policy interactions,
- cheaper to run,
- easier to bound,
- and often sufficient for small tasks.

This is not anti-agentic skepticism. It is task-relative design discipline.

Scope boundaries

This book deliberately stops short of several neighboring areas.

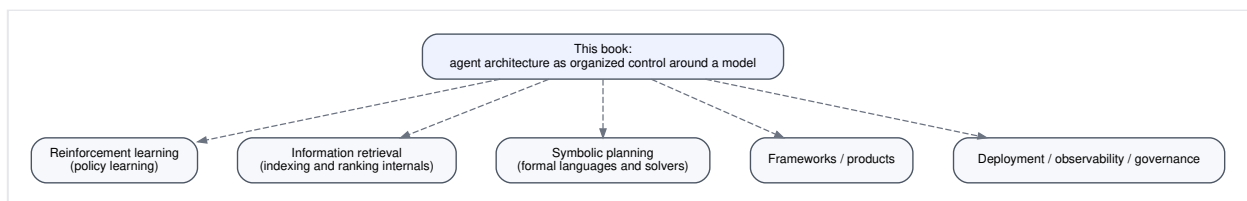


Figure 9.2. Scope boundaries. This book focuses on architectural organization around a generative model, not on policy learning, retrieval internals, formal planners, frameworks, or production operations.

- **Reinforcement learning** asks how policies can be learned from reward over time (Sutton and Barto, 2018). This book instead asks how bounded control can be architected when the policy is largely specified, prompted, or hand-designed.
- **Information retrieval** asks how to index, rank, and evaluate retrieval systems (Manning et al., 2008). This book treats search as a conceptual tool interface and does not go inside ranking algorithms.
- **Symbolic planning** studies formal representations, operators, and search procedures (Russell and Norvig, 2021). This book borrows planning intuition without teaching formal planning languages or solvers.

- **Software patterns, frameworks, and orchestration products** concern implementation vehicles. This book stays at the architecture-pattern level.
- **Operations and production engineering** concern deployment, observability, governance, and failure analysis in running systems. Those are important, but they are outside the present conceptual scope.

Marking these boundaries is pedagogically useful. The reader does not need to master all neighboring fields before learning how agent architectures are put together.

Worked example: selecting a pattern by task shape

Let us apply the six dimensions explicitly.

Running literature-review task

- Horizon: medium
- Uncertainty: moderate; evidence quality changes the path
- Memory need: moderate; notes and matrix state matter
- Tool dependence: high; retrieval and reading are necessary
- Feedback richness: high; support checks and contradictions matter
- Autonomy bound: bounded but nontrivial

Recommended pattern: bounded plan-memory-act-reflect-stop loop.

Short survey-finding task

- Horizon: short
- Uncertainty: low to moderate
- Memory need: low
- Tool dependence: moderate
- Feedback richness: limited
- Autonomy bound: low

Recommended pattern: prompt-only with retrieval, or scripted retrieval workflow.

This is the kind of explanation the book aims to make second nature.

Pseudocode example

```

procedure SELECT_ARCHITECTURE(task_shape):
  if task_shape.horizon is short
    and task_shape.uncertainty is low
    and task_shape.memory_need is low:
    return prompt_or_scripted

  if task_shape.tool_dependence is high
    and task_shape.feedback_richness is moderate_or_high
    and task_shape.autonomy_bound is bounded:
    return bounded_adaptive_loop

  if task_shape.requires_structured_evidence_accumulation:
    return bounded_plan_memory_act_reflect_stop

  return simplest_architecture_that_meets_constraints

```

The final line is the most important. Simplicity is not a fallback. It is often the correct answer.

Cumulative artifact update

The literature-review assistant now gains its architecture comparison matrix.

Artifact	Current content
Comparison matrix	prompt-only vs scripted workflow vs bounded plan-memory-act-reflect-stop loop
Selection lens	horizon, uncertainty, memory need, tool dependence, feedback richness, autonomy bound

This chapter turns component literacy into judgment.

What breaks if this component is missing or misused?

Without comparison discipline, architecture choice drifts toward fashion. Designers overbuild because richer diagrams look impressive, or underbuild because a powerful model seems sufficient.

Misuse takes several forms:

1. **buzzword selection:** choosing the architecture with the best name rather than the best fit,
2. **component maximalism:** including every possible module “just in case,”
3. **neighbor confusion:** assuming RL, formal planning, or IR internals are prerequisites rather than adjacent specializations,
4. **framework drift:** talking about software products instead of control structure.

Failure modes, limitations, and misconceptions

- **One-size-fits-all design.** No single architecture dominates across all task shapes.
- **Choosing by buzzword.** Terms like “agent,” “planner,” or “memory” do not justify themselves.

- **Treating neighboring fields as prerequisites.** They are neighbors, not gates (Sutton and Barto, 2018; Manning et al., 2008; Russell and Norvig, 2021).
- **Drifting into frameworks or ops.** Implementation details matter later, but they are not the focus of architectural comparison here.

Chapter summary

Architectural judgment comes from matching control structure to task shape. The richest bounded loop is appropriate for the running literature-review assistant because the task has moderate horizon, tool dependence, evidence accumulation, and bounded autonomy. Simpler architectures remain better fits for shorter, more stable tasks. The chapter also marked the topic's boundaries relative to RL, IR, symbolic planning, frameworks, and operations.

The final chapter now uses the accumulated artifacts to synthesize and defend a literature-review assistant from scratch.

References and further reading

- (Xi et al., 2023) for a broad overview of LLM-based agent patterns.
- (Russell and Norvig, 2021) for neighboring background on search and planning.
- (Sutton and Barto, 2018) for policy learning as a different problem family.
- (Manning et al., 2008) for retrieval internals beneath search tools.

Architecture delta

- **Added:** a task-shape lens for comparing whole architectures.
- **Failure addressed:** selecting architectures by fashion or isolated components.
- **New risk introduced:** false precision if the comparison matrix is treated as a formula rather than a judgment aid.

Chapter 10 — Designing the Literature-Review Assistant

AA-S09 — Capstone synthesis

Design pressure from the running case

The book's recurring literature-review assistant has been incrementally upgraded chapter by chapter. The final pressure is synthesis: can we now assemble a coherent bounded architecture and defend every inclusion, omission, and trade-off?

A weak capstone would simply paste together every component the book mentioned. A stronger capstone asks harder questions:

- Which components are essential for this specific task?
- Which are optional but useful?
- Which would be overbuilding?
- Can we trace one run from request to stop or handoff and show that the architecture actually hangs together?

This chapter answers those questions by producing two designs: a **minimal adequate architecture** and a **richer defended architecture**.

Chapter overview

The capstone has five jobs:

1. restate the bounded task specification,
2. propose a minimal adequate architecture,
3. propose a richer architecture and justify the additions,
4. walk through a full execution trace,
5. predict a failure and revise the design.

The result is not a framework tutorial. It is a design memo for a bounded agent architecture.

Nearest confusable distinction — minimal adequate architecture vs overbuilt architecture

A minimal adequate architecture contains the smallest set of components required by the task specification. An overbuilt architecture contains extra mechanisms whose control cost, maintenance burden, or failure surface exceeds their value for the bounded task.

Design memo: task specification

We begin with the compiled task specification from earlier chapters.

```
G_review = <objective = produce a 1,200-word literature review on citation-grounded question-answering assistants for students; constraints = 2021–2024, 8–12 papers if available, architecture-first framing, comparison matrix required, no
```

vendor-specific tutorials; success criteria = each substantive claim supported by at least one read source, at least three architecture patterns compared if evidence allows, uncertainty explicitly marked where evidence is mixed or thin; stop/handoff conditions = ask if scope is materially ambiguous, stop when criteria are met with adequate evidence, hand off if evidence is too thin or interpretive conflict exceeds system authority>.

The task is bounded, information-centric, and moderately uncertain. It is not open-ended autonomous research. It is not production deployment. It therefore needs a selective architecture, not maximal machinery.

Minimal adequate architecture

The **minimal adequate version** includes the smallest set of components that can plausibly satisfy `G_review`.

Included components

1. **Explicit task specification** `G_review`.
2. **Run-local external state** `S_t` with candidate sources, screened set, notes, matrix rows, and completion markers.
3. **Conceptual tool interfaces** for search, document read, note write, and citation check.
4. **Simple plan** as a checklist: search → screen → read → synthesize → verify → stop.
5. **Verification checks** for claim support and completion.
6. **Stop/clarification logic** for ambiguity, thin evidence, and sufficient completion.

Omitted components

1. **Persistent semantic memory** beyond curated procedure templates.
2. **Rich episodic cross-session memory**.
3. **Elaborate branching search**.
4. **Multi-agent decomposition**.
5. **Open-ended autonomous goal generation**.

Those omissions are deliberate. The task does not require them to be understood or executed well at a bounded conceptual level.

Why this version is minimally adequate

The task requires evidence accumulation and bounded synthesis, so prompt-only interaction is insufficient. It also requires some adaptation, so a rigid script is risky. But the task does not demand long-term personalization, autonomous tool discovery, or learned policy optimization. Therefore the minimal adequate design is a **bounded adaptive loop with light planning and strong verification**.

Richer defended architecture

The **richer version** adds only those elements that materially improve robustness for the running case.

Added components

1. **Curated procedure memory.** A reusable note schema, screening checklist, and stop checklist.
2. **Scoped episodic query history.** Recent query attempts and their observed quality within the current topic family.
3. **Replanning loop.** Triggered when evidence is broader, sparser, or more contradictory than expected.
4. **Contradiction register.** A dedicated structure for tracking where source notes disagree.
5. **Explicit handoff packet.** If the system stops without full resolution, it hands the user a packet containing scope assumptions, evidence gathered, unresolved disagreements, and next recommended actions.

These additions are defended because they directly respond to failures from earlier chapters: repeated query mistakes, stale or missing procedure structure, and weak handling of disagreement.

Full architecture diagram

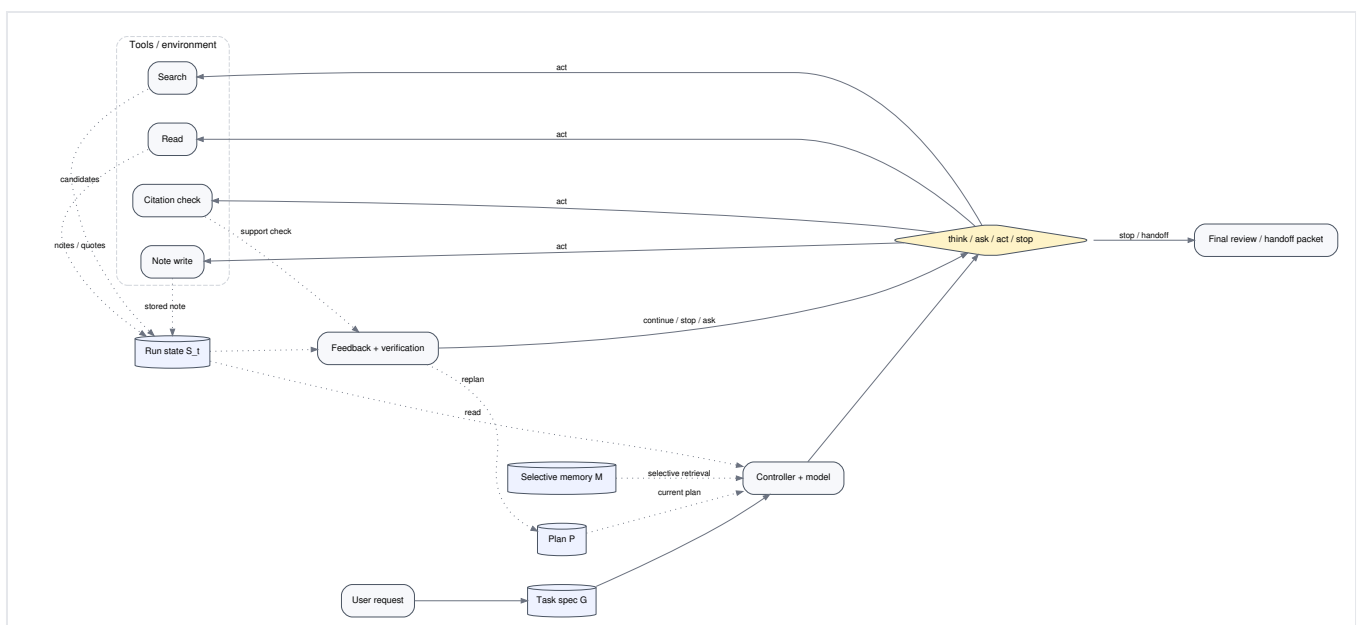


Figure 10.1. Full bounded architecture for the literature-review assistant. The model sits inside a goal-conditioned loop with explicit state, selective memory, tool interfaces, planning, verification, and stop or handoff logic.

The diagram can be read from left to right and back again:

- `G_review` constrains the whole run.
- The controller maintains `S_t`.
- The model helps choose among think, ask, act, and stop.
- Tools couple the system to the environment.
- Verification and feedback feed back into the controller.
- Stop or handoff logic ends the run within bounds.

Component inclusion rationale

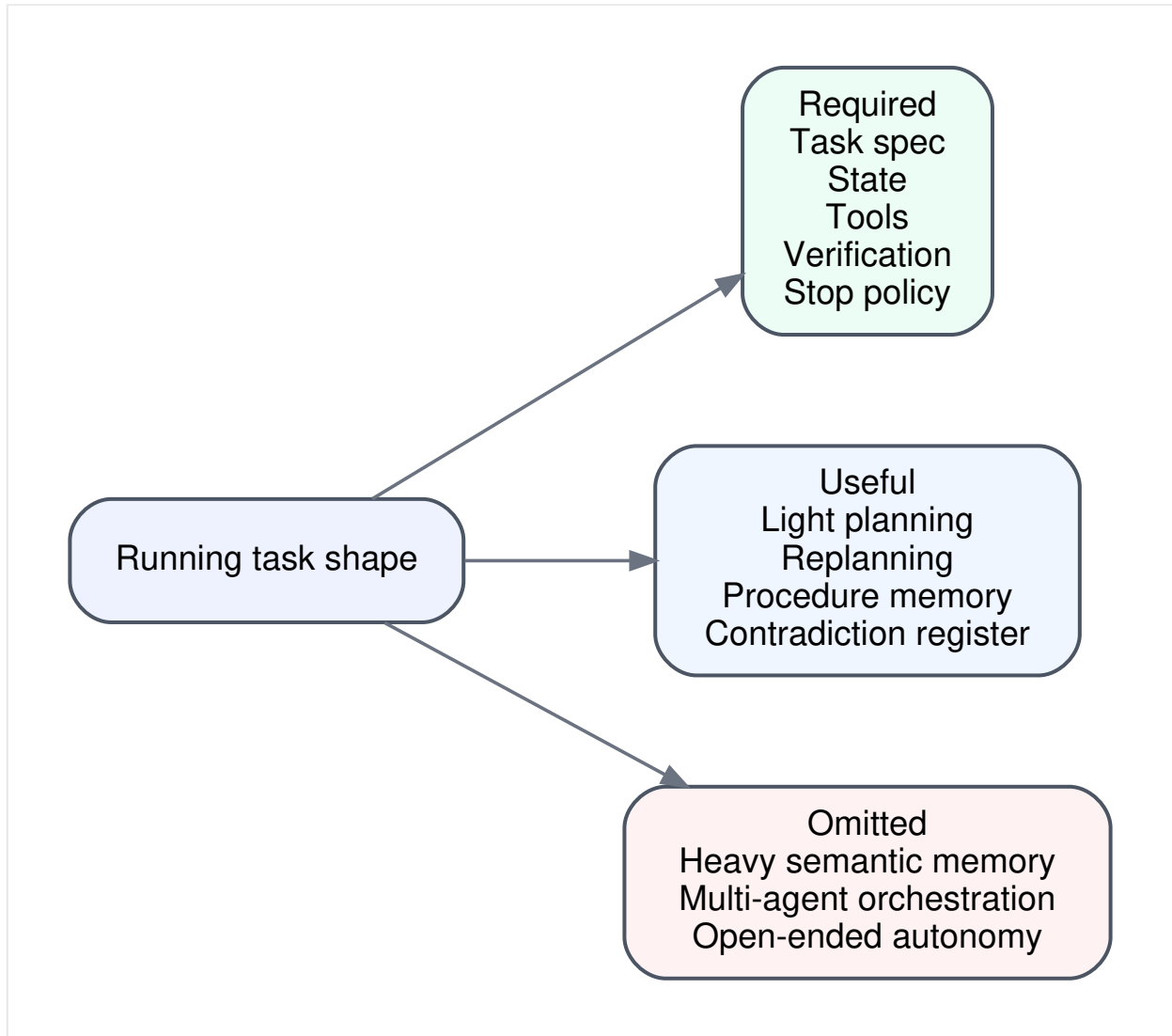


Figure 10.2. Inclusion rationale. Components are present because the task shape demands them, not because a generic “agent stack” says they should be there.

The rationale can be summarized in Table 10.1.

Component	Included?	Why
Explicit task spec	yes	without it the system cannot evaluate success or stopping
Run-local state	yes	the review needs candidate tracking, notes, matrix rows, and completion markers
Persistent semantic memory	limited	concept reuse helps, but aggressive persistence risks contamination
Tool interfaces	yes	fresh evidence is necessary
Planning	yes, but light	the task has dependencies and moderate uncertainty
Replanning	yes	observations often change which papers or categories matter
Generic reflection	limited	

Component	Included?	Why
		useful only after evidence-bearing checks
Verification	yes	support mapping is central to the task
Stop/handoff policy	yes	bounded autonomy requires principled endings
Multi-agent orchestration	no	unnecessary for the bounded task shape

Worked execution trace

The following trace shows one plausible run of the richer design.

t	goal view	context	external state	observation	action	result	stop decision
0	compile bounded review task	raw request + default assumptions	empty candidate, note, and matrix stores	none	think	G_review instantiated ; checklist loaded	continue
1	determine if scope is clear	G_review + ambiguity tests	no clarification yet	phrase “for students” could mean two subdomains	ask	user clarifies: include study/tutoring assistants only if they present citations or quotes in final answers	continue
2	gather candidate sources	clarified G_review + initial plan	query history empty	search returns 18 candidates, many generic RAG systems	act	candidate list stored; broad-result flag raised	continue
3	refine search and screening	broad-result flag + current candidates	screening rule draft exists	abstracts suggest answer-level citations are rarer than simple retrieval	think	exclusion rule tightened; plan revised toward quote verification and inline citation patterns	continue
4	read representative sources	accepted candidates + note schema	screened set updated	first three papers reveal two distinct grounding patterns and one weakly	act	structured notes stored; matrix schema gains “evidence	continue

t	goal view	context	external state	observation	action	result	stop decision
				relevant system		presentation" column	
5	verify support for provisional synthesis	notes + provisional summary	contradiction register empty	citation check finds two draft claims insufficiently supported	act	unsupported claims flagged; one sentence softened, one removed	continue
6	resolve remaining uncertainty	notes + contradiction tests	four strong notes, two uncertain notes	one source disagrees about educational trust benefit	think	contradiction register created; uncertainty section added to plan	continue
7	decide completion	summary draft + matrix + support map	success markers nearly complete	success criteria met for architecture review; trust-effect section remains mixed but explicitly marked	stop	final review produced with uncertainty note and no overclaiming	stop complete

This trace is deliberately bounded. It does not try to read everything. It shows a coherent run whose actions can be justified from `G_review`, `S_t`, tool observations, and stop policy.

Pseudocode example: the defended richer loop

```

procedure RUN_LITERATURE_REVIEW(request):
  G <- compile_task_spec(request)
  S <- init_state(G)
  P <- init_light_plan(G)
  M <- load_procedure_templates()

  while true:
    O <- observe_latest(S)
    C <- build_context(G, S, P, O, M)
    A <- choose_next_move(C)      # think | ask | act | stop

    if A == ask:
      user_reply <- request_clarification(C)
      S <- update_with_user_reply(S, user_reply)
      P <- maybe_replan(P, S)

    else if A == act:
      result <- invoke_tool_or_check(C)
      S <- update_with_result(S, result)
      if replan_triggered(P, S, result):
        P <- revise_plan(P, S)

    else if A == think:
      S <- update_control_state(S, C)
      if replan_triggered(P, S, O):
        P <- revise_plan(P, S)

    else:
      return finalize_or_handoff(G, S, P)

  feedback <- run_support_contradiction_completion_checks(S, G)
  S <- apply_feedback(S, feedback)

  if should_stop(G, S, feedback):
    return finalize_or_handoff(G, S, P)

```

This loop is still conceptually small. The extra sophistication lies not in algorithmic complexity but in the number of architectural distinctions it keeps separate.

Minimal adequate version vs richer version

The difference between the two capstone designs is worth stating explicitly.

Dimension	Minimal adequate	Richer defended
Memory	run-local state plus procedure template	adds scoped episodic query history and curated reusable procedures
Planning	one initial checklist	explicit replanning triggers
Feedback	support check and completion check	adds contradiction register and richer feedback packet
Handoff	simple stop with caveat	explicit handoff packet with unresolved issues and next actions
Risk	may repeat weak queries or under-handle disagreement	more robust but more complex to govern

The richer version is justified for the running case because the task often benefits from query reuse, contradiction tracking, and structured unresolved-output handling. But the minimal version remains a valid design for a narrower review request.

Predicted failure mode and concrete revision

A strong architecture design memo should predict where failure is still likely.

Predicted failure

The richer version may still under-handle **near-duplicate evidence**. Suppose several sources cite the same underlying benchmark or claim. The system may treat them as independent support and overestimate consensus.

This is a structural failure, not just a citation-format issue. The architecture tracks sources and contradictions, but not evidence dependence.

Revision

Add a lightweight **evidence lineage field** to each note:

- whether the claim is directly observed in the paper,
- inherited from another cited source,
- or repeated from common benchmark reporting.

Then revise the support check so that multiple notes based on the same underlying claim source do not automatically count as independent support.

This is a good capstone revision because it preserves the bounded character of the system while improving its error analysis discipline.

Failure-mode audit of the final design

The book's light structural evaluation returns in this final audit.

Structural area	Current defense	Residual vulnerability
Goal representation	explicit <code>G_review</code>	hidden user expectations may still remain
State	explicit candidate, screening, note, matrix, and completion fields	poor pruning could clutter state
Memory	selective and scoped	stale curated concepts could still leak in
Planning	light plus replanning	overactive replanning remains possible
Action/tool use	explicit contracts	tool results can still be noisy or incomplete
Grounding	support mapping and contradiction checks	evidence dependence may be underestimated
Stop/handoff	explicit thresholds and handoff packet	thresholds remain task-relative and judgment-based

The important point is not that the design has become failure-proof. It is that failures can now be anticipated in architectural terms.

Specialization pathways beyond this book

This capstone closes the book's main conceptual arc, but it also points outward.

A reader who wants to go deeper might specialize in:

- **memory-heavy systems**, including retrieval design and long-context management,
- **planning**, including more formal search or decomposition methods,
- **evaluation**, including benchmark design and quantitative methodology,
- **human-AI interaction**, including clarification and handoff design,
- **reinforcement learning**, for learned policy improvement,
- **information retrieval**, for search and ranking internals,
- **symbolic planning**, for formal operators and planners,
- **production engineering**, for deployment, observability, governance, and operations.

Those are real next steps. They are not prerequisites for understanding the architectural spine built in this book.

Cumulative artifact update

The cumulative artifacts now form one coherent bounded architecture:

- `G_review` task specification,
- trace schema and state inventory,
- memory policy,
- tool contracts,
- plan and replanning rules,
- feedback and verification rules,
- stop/handoff policy,
- architecture comparison matrix,
- final minimal and richer architecture designs.

What breaks if this component is missing or misused?

In a capstone design, the most common misuse is **template copying**. The designer pastes in memory, planning, reflection, and tools without a task-relative reason.

For the running case, three omissions would be especially damaging:

1. **No stop/handoff policy.** The system would keep searching or overclaim under thin evidence.
2. **No memory policy.** Notes and procedure knowledge would blur together.
3. **No verification path.** The architecture could produce polished but weakly supported synthesis.

The capstone discipline is therefore inclusion and omission together. A good design says what is absent and why.

Failure modes, limitations, and misconceptions

- **Copying a stack diagram without task fit.** Architecture must answer to task shape.
- **Including every possible component.** More boxes do not imply better control.
- **Forgetting stop/handoff in the final design.** Many otherwise good diagrams still fail here.
- **Presenting a diagram without a run trace.** A defensible architecture should be able to explain one bounded run from request to stop.

Chapter summary

This chapter turned the book's distinctions into a defended design for a bounded literature-review assistant. The final architecture is not defined by vendor tools, repositories, or deployment choices. It is defined by goal representation, control loop, state placement, memory policy, tool contracts, planning, grounding, feedback, and stop or handoff logic. The capstone also showed that a good design memo predicts failure and revises the architecture accordingly.

The book's central spine question can now be answered clearly: **a generative model becomes a goal-directed agentic system when it is embedded in an architecture that represents the task explicitly, carries the right state, selects actions adaptively from observations, grounds its claims in evidence, and knows when to stop or hand off.**

References and further reading

- (Yao et al., 2022; Schick et al., 2023; Nakano et al., 2021) for concrete systems that combine generation with action and evidence gathering.
- (Park et al., 2023; Packer et al., 2023) for memory-oriented architectural ideas.
- (Madaan et al., 2023; Shinn et al., 2023) for reflection-oriented methods and their limits.
- (Xi et al., 2023) for a broad map of LLM-based agent research beyond the bounded design developed here.

Architecture delta

- **Added:** a complete defended architecture and a trace that makes it operational.
- **Failure addressed:** isolated component knowledge without whole-system synthesis.
- **New risk introduced:** overconfidence that one capstone design should generalize unchanged to every task.

Bibliography

1. **Saleema Amershi, Daniel S. Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, et al.** (2019). "Guidelines for Human-AI Interaction." Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems. [URL](#)
2. **Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, et al.** (2020). "Language Models are Few-Shot Learners." Advances in Neural Information Processing Systems 33. [URL](#)
3. **Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, et al.** (2022). "Inner Monologue: Embodied Reasoning through Planning with Language Models." arXiv preprint arXiv:2207.05608. [URL](#)
4. **Wenlong Huang, Pieter Abbeel, Deepak Pathak, Igor Mordatch, et al.** (2022). "Language Models as Zero-Shot Planners: Extracting Actionable Knowledge for Embodied Agents." arXiv preprint arXiv:2201.07207. [URL](#)
5. **Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, et al.** (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." Advances in Neural Information Processing Systems 33. [URL](#)
6. **Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, et al.** (2023). "Self-Refine: Iterative Refinement with Self-Feedback." arXiv preprint arXiv:2303.17651. [URL](#)
7. **Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schutze** (2008). *Introduction to Information Retrieval*. Cambridge University Press. DOI: [10.1017/CBO9780511809071](#) [URL](#)
8. **Jacob Menick, Maja Trebacz, Vladimir Mikulik, John Aslanides, Francis Song, Martin Chadwick, et al.** (2022). "Teaching Language Models to Support Answers with Verified Quotes." arXiv preprint arXiv:2203.11147. [URL](#)
9. **Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, et al.** (2021). "WebGPT: Browser-assisted Question-Answering with Human Feedback." arXiv preprint arXiv:2112.09332. [URL](#)
10. **Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, et al.** (2023). "MemGPT: Towards LLMs as Operating Systems." arXiv preprint arXiv:2310.08560. [URL](#)
11. **Raja Parasuraman, Thomas B. Sheridan, and Christopher D. Wickens** (2000). "A Model for Types and Levels of Human Interaction with Automation." IEEE Transactions on Systems, Man, and Cybernetics-Part A: Systems and Humans. Vol. 30(3). pp. 286-297. DOI: [10.1109/3468.844354](#) [URL](#)
12. **Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, Michael S. Bernstein, et al.** (2023). "Generative Agents: Interactive Simulacra of Human Behavior." Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology. [URL](#)
13. **Stuart Russell and Peter Norvig** (2021). *Artificial Intelligence: A Modern Approach*. 4th ed. Pearson. [URL](#)

14. **Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, et al.** (2023). “Toolformer: Language Models Can Teach Themselves to Use Tools.” *Advances in Neural Information Processing Systems* 36. [URL](#)
15. **Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, Shunyu Yao, et al.** (2023). “Reflexion: Language Agents with Verbal Reinforcement Learning.” *arXiv preprint arXiv:2303.11366*. [URL](#)
16. **Richard S. Sutton and Andrew G. Barto** (2018). *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press. [URL](#)
17. **Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, et al.** (2023). “Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models.” *arXiv preprint arXiv:2305.04091*. [URL](#)
18. **Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwon Ding, Boyang Hong, et al.** (2023). “The Rise and Potential of Large Language Model Based Agents: A Survey.” *arXiv preprint arXiv:2309.07864*. [URL](#)
19. **Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, et al.** (2022). “ReAct: Synergizing Reasoning and Acting in Language Models.” *arXiv preprint arXiv:2210.03629*. [URL](#)
20. **Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, et al.** (2023). “Tree of Thoughts: Deliberate Problem Solving with Large Language Models.” *arXiv preprint arXiv:2305.10601*. [URL](#)